



ArtComputer

DESIGNER User Manual

Automated Test with the Robot

Designer User Manual

Contents

Working with Proxy	6
Default certificate selection.....	8
Configuration for a default certificate	8
Working with a certificate file (security)	9
Configuration for a .p12 or .pfx certificate	9
Configuration for a .crt certificate	9
Login	10
User Interface	10
User interface for the editing	11
User Interface	11
Control Panel.....	12
User Interface	12
Scenario	13
User Interface	13
Scenario Edit	14
Scenario Import.....	15
Test.....	16
Test Edit	21
Import Test.....	23
Logfile.....	24
Gallery	25
Suite Set	26
User Interface	26
Suite Set Edit	27
Suites	28
User Interface	28
Suites Set Edit	29
Logfile.....	30
Gallery	31

Story Set	32
Story set	32
Story set Edit	33
Stories	34
Stories Edit	35
Logfile	36
Gallery	37
Reference	38
Reference	38
Reference Edit	39
Export Reference	40
Import Reference	41
Specific Reference	42
Dictionary	43
Dictionary set	43
Dictionary set Edit	44
Dictionary word	45
Dictionary word Edit	46
Dictionary unused word	47
Dictionary scan web element	48
Rules	51
Rules set	51
Rules set Edit	53
Rules	54
Rules Edit	55
Import rule set (including rules)	56
Loop	57
Principle	57
Break a loop	58
End of a loop	59
Examples of loop	60
Performance	62
Principle	62

Define the measure	63
Mechanism.....	64
Performance with the Robot	66
Performance with Admin right	70
Example of graphs of performance designed in Excel	71
Testing mobiles (web browser).....	72
Android Studio - Installation	72
Testing mobiles (Apps)	77
Android Studio - Installation	77
Android Studio - Scanner	79
Source: server.js.....	81
Source: index.html	83

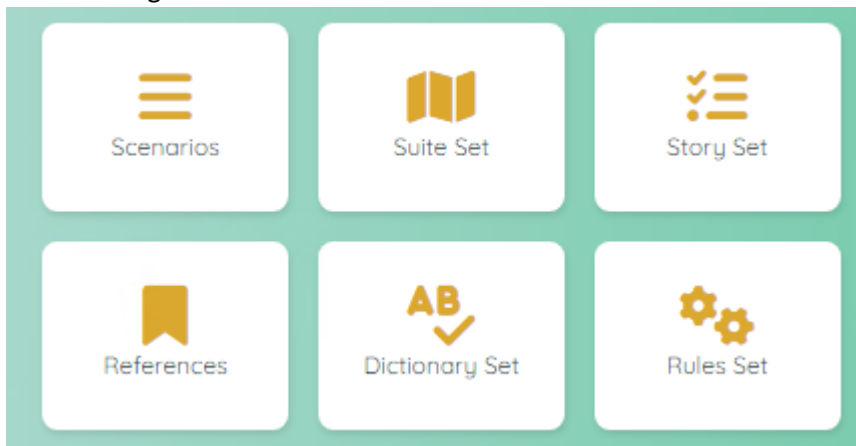
Foreword

This user manual will give you information on how to use the robot as a Designer.

Designer:

The Designer is responsible for:

- The design of the Scenarios
- The design of the Suites
- The design of the Stories
- The management of the References
- The management of the Dictionary
- The design of the Rules

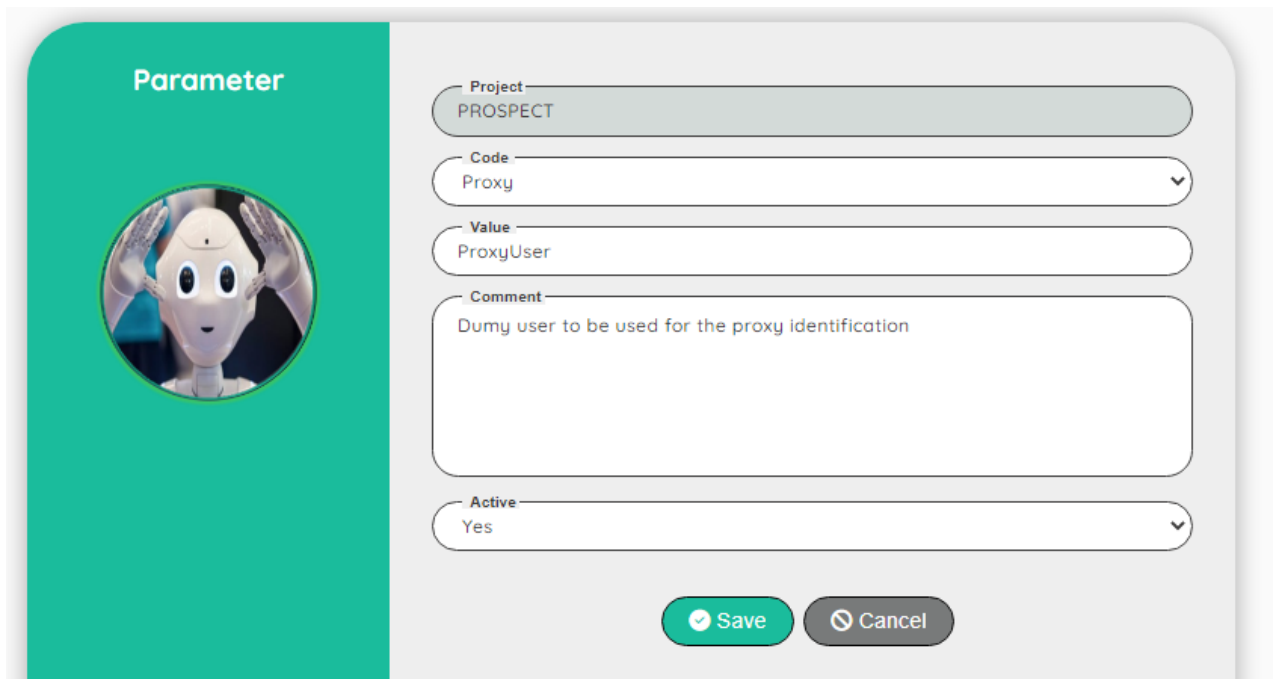


Working with Proxy

If you need to test a project that requires a proxy, please use the following method.

Job to be performed by the Administrator:

In the parameters of the project, create a new entry for the parameter Proxy



The screenshot shows a 'Parameter' configuration window. On the left, a teal vertical bar contains the title 'Parameter' and a circular icon of a robot head. The main area is a light gray form with the following fields:

- Project:** A text field containing 'PROSPECT'.
- Code:** A dropdown menu with 'Proxy' selected.
- Value:** A text field containing 'ProxyUser'.
- Comment:** A text area containing 'Dummy user to be used for the proxy identification'.
- Active:** A dropdown menu with 'Yes' selected.

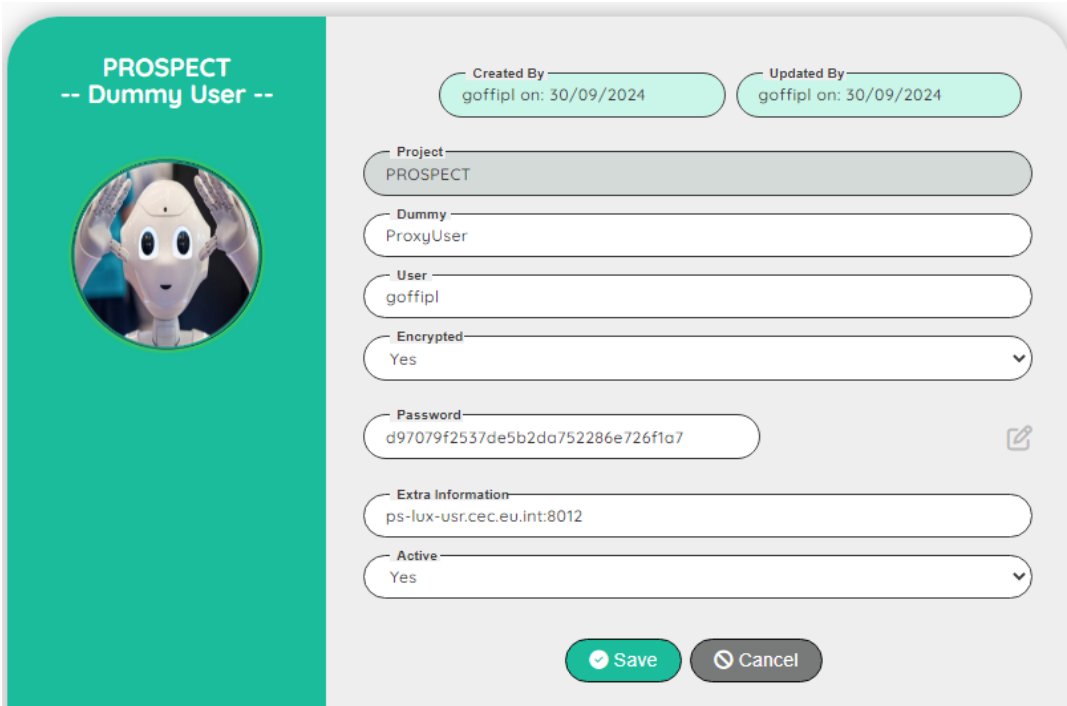
At the bottom of the form are two buttons: a green 'Save' button with a checkmark icon and a gray 'Cancel' button with a close icon.

The value must be a valid dummy user

Job to be performed by the Tester (or Designer, Administrator):

Go to the dummy users and add a new entry for the value defined in the parameter (here: ProxyUser)

In the field Extra information, fill the proxy link including the port number



The screenshot shows a configuration window for a 'Dummy User' in the PROSPECT application. On the left, there is a teal header with the text 'PROSPECT -- Dummy User --' and a circular profile picture of a robot head. The main area contains several input fields and dropdown menus:

- Created By:** goffipl on: 30/09/2024
- Updated By:** goffipl on: 30/09/2024
- Project:** PROSPECT
- Dummy:** ProxyUser
- User:** goffipl
- Encrypted:** Yes (dropdown menu)
- Password:** d97079f2537de5b2da752286e726f1a7 (with a copy icon)
- Extra Information:** ps-lux-usr.cec.eu.int:8012
- Active:** Yes (dropdown menu)

At the bottom, there are two buttons: 'Save' (with a checkmark icon) and 'Cancel' (with a close icon).

Note: You can encrypt the password

If you don't need a proxy login, you don't need to define a specific parameter and dummy user.

If you try to connect to a proxy website without defining the parameter and the dummy user, the application will display a window to ask you to enter a proxy login/password.

This popup window cannot be manipulated by normal functions!

If you define a proxy, the Robot will be able to login to a normal website or a website with proxy. For information: the time to launch the script will be shorter with a proxy defined!

Default certificate selection

With Chrome (and only with Chrome), you can define the certificate to select by default.

Note: even if you select a certificate by default, the secret code will be required!

However, if you execute different scenarios in a suite, the secret code needs to be keyed only once!

The selection of the certificate by default can be configured with the Environment values

Configuration for a default certificate

Action	Role	Comment
Create an Environment: CertificateRole	Designer	ISSUER or SUBJECT (*)
Create an Environment: CertificateValue	Designer	E.g.: Citizen CA

(*) Only these two values are allowed !

I recommend to use **ISSUER** because the value can be a generic name like "Citizen CA".

If you need to select a very specific certificate, you can use **SUBJECT** with for instance the name of the person: "Mergo D'Heer Alaune (Authentication)"

It's very important that the filter you create with the role and the value returns only one certificate. Otherwise, a popup windows will be displayed to ask you to choose the right certificate!

To show your certificates, you can open **Chrome**, go to Settings, Privacy & Security, Security, Manage Certificates and finally "Your Certificates"

Check the certificates with (Authentication), click on the eye icon.

In Details, the first info can be used for the **SUBJECT** value (Certificate Hierarchy)

In the middle box (Certificate Fields), click on Issuer. The Field Value starting with "CN =" can be used for the **ISSUER** value.

I suggest you define a dataset containing the Role and Value.

Then, you create a simple scenario to retrieve the Role and Value from this dataset (getData) and store them in a Reference (setReference).

This method offers you great flexibility for certificate selection.

You can even create a story within this scenario to allow the tester to choose the certificate for testing.

Working with a certificate file (security)

You can use the following certificates: .p12, .pfx and .crt

If you need to test a project that requires a certificate to perform an authentication, you have to do the following task:

- Upload the right certificates (certificate and key)
- Define reference to define the full certificate behaviour

Configuration for a .p12 or .pfx certificate

Action	Role	Comment
Upload the .p12 or .pfx certificate	Admin	Use the Upload menu
Create an Environment: CertificateUrl	Designer	E.g.: https://myDossier.rn
Create an Environment: Certificate	Designer	E.g.: myCertificate.p12
Create an Environment: CertificateCode	Designer	E.g.: 1234

Configuration for a .crt certificate

Action	Role	Comment
Upload the .crt certificate	Admin	Use the Upload menu
Upload the .key certificate	Admin	Use the Upload menu
Create an Environment: CertificateUrl	Designer	E.g.: https://myDossier.rn
Create an Environment: Certificate	Designer	E.g.: myCertificate.crt
Create an Environment: CertificateCode	Designer	Optional , only if the key file is encrypted, otherwise use <N/A>

As the certificate is used at the very beginning of the test, if there is an error, you will not be notified, you have to check on the console (RobotVx) to view the error message.

If something goes wrong, please check the configuration steps below!

I suggest you define a dataset containing all the required values.

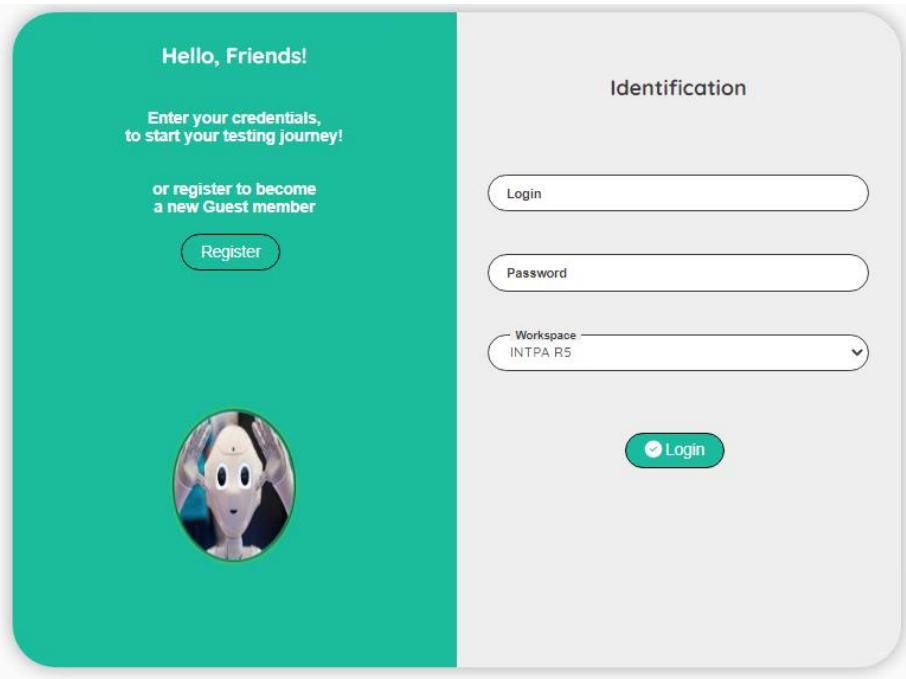
Then, you create a simple scenario to retrieve the data from this dataset (getData) and store them in a Reference (setReference).

Login

User Interface

To access to the Robot, you need to have a login.

After you successfully register to the application as a new user (Guess), you have to wait that the Administrator will assign you a Role to a project (or multiple projects).



The interface is divided into two main sections. The left section has a teal background and contains the text 'Hello, Friends!', 'Enter your credentials, to start your testing journey!', and 'or register to become a new Guest member'. Below this text is a 'Register' button and a circular profile picture of a robot. The right section has a light gray background and is titled 'Identification'. It contains three input fields: 'Login', 'Password', and 'Workspace' (which is a dropdown menu currently showing 'INTPA R5'). Below these fields is a green 'Login' button with a white eye icon.

Topic	Icon	Comment
Register		Click to Register as a new user
Login		Enter your login
Password		Enter your password
Workspace		Enter your workspace
Login		Click to login to the Robot

User interface for the editing

User Interface

We use the same user interface when editing a record.

To avoid repeating the same explanation all the time, you will find here a generic explanation on how to use the edit interface.

When you have the permission to edit a record, you will see the following icons:



Topic	Icon	Comment
Delete	 or  	Delete a record (a confirmation is required!) To delete multiple records, select the records first.
Edit		Go to a screen to update the record.
Copy	 	First select the record to copy and then click on the copy icon. Record(s) will be set after the copy icon.
Move	 	First select the record to move and then click on the move icon. Record(s) will be set after the move icon.
Select / Unselect	 / 	Select or unselect a record

Depending of the context, some extra icon(s) can be available.

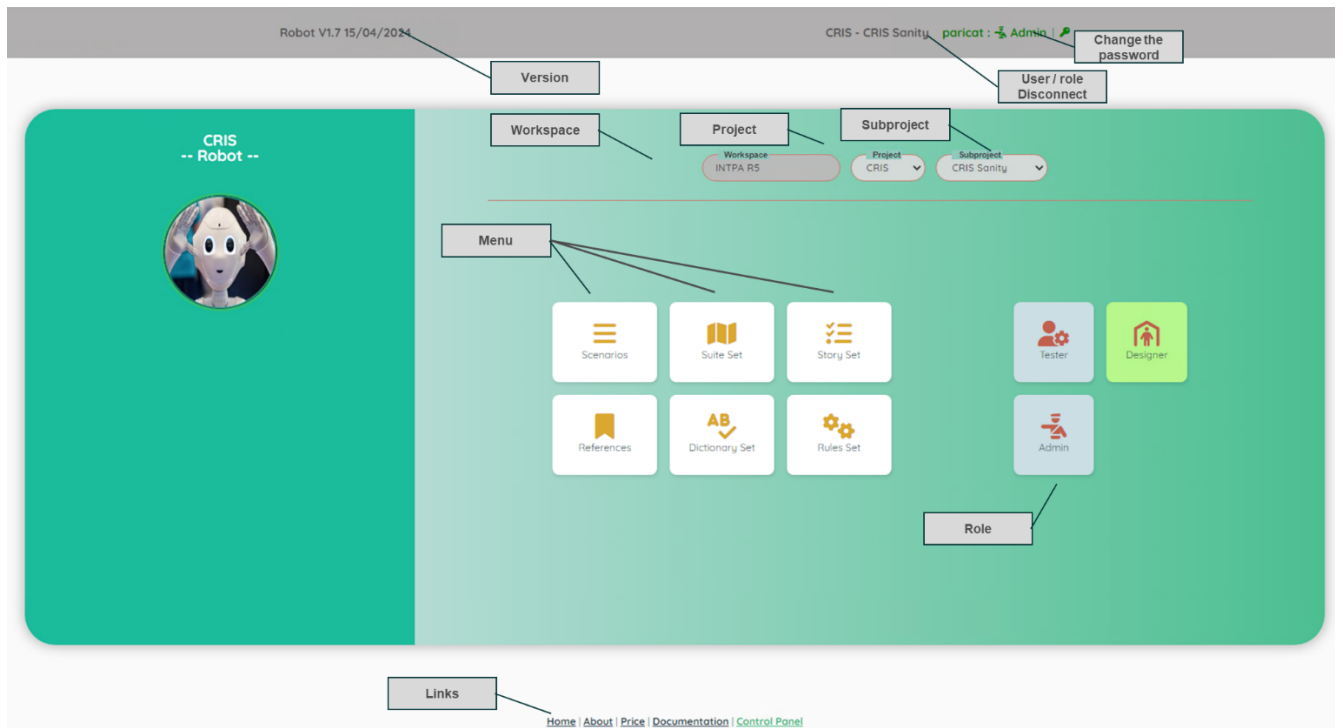
The extra icon(s) will be explain in the section dedicated to the context.

Control Panel

User Interface

The dashboard will allow you to execute pre-defined stories.

If you are assign to a **DESIGNER** or **ADMIN** role, your user interface will look like this:

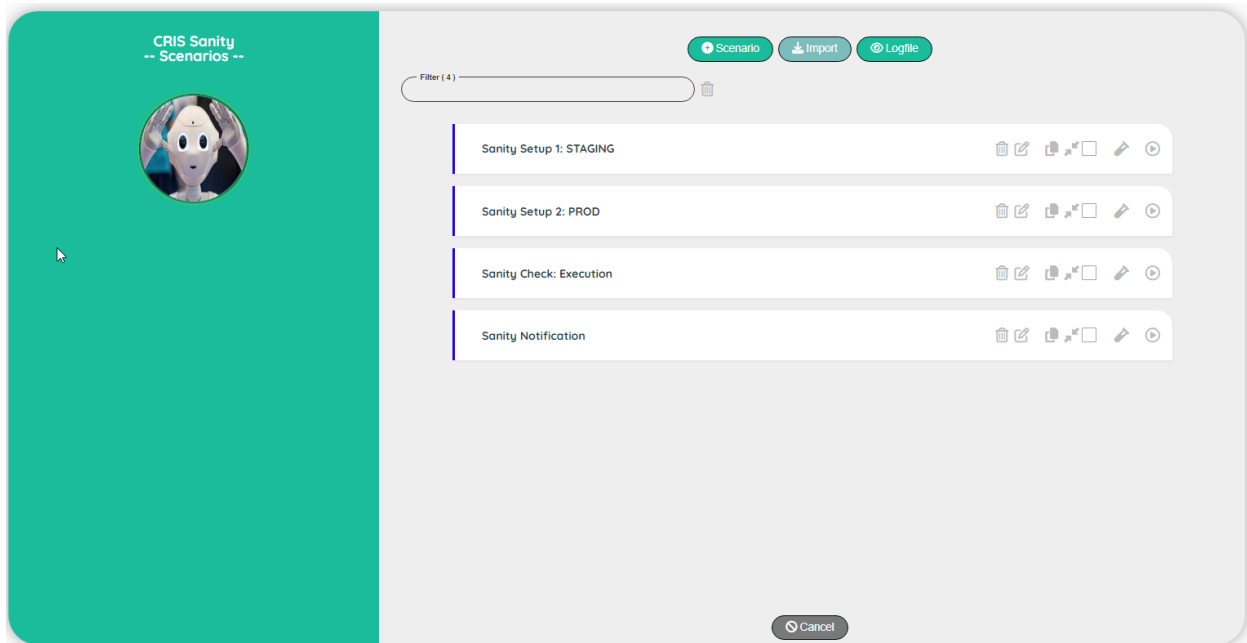


Topic	Comment
Version	Version of the Robot.
Workspace	For info: the name of the workspace of the customer.
Project	List of the project(s) that you have access (assignment to a project is done by an Administrator).
Subproject	Access to all the subproject(s) of the selected project.
	When executing a story, you can stop the execution.
User/Role Disconnect	For info: your login and your role. If you click on the 'User/Role', you will be disconnected from the application.
Change the password	Click on the icon to change your current password
Menu	Click on the items to open a new section of the application
Links	Useful links

Scenario

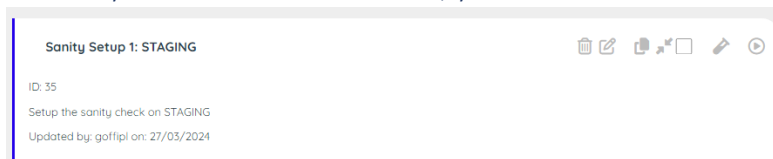
User Interface

The Scenario will allow you to manage the scenarios and the tests



Topic	Icon	Comment
Test		Go to the screen Test.
Execute		Execute a scenario.
Add Scenario		Add a new scenario at the end of the list.
Cancel		Back to the Dashboard screen.
Logfile		Go to the screen to view the Log file.
Import		Import a scenario from any project (same workspace).

Note: if you click on the comment, you will see extra information



Scenario Edit

CRIS

-- Scenario --



Created By

goffipl on: 27/03/2024

Updated By

goffipl on: 27/03/2024

Subproject

CRIS Sanity

Scenario

Sanity Setup 1: STAGING

Comment

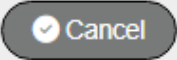
Setup the sanity check on STAGING

Active

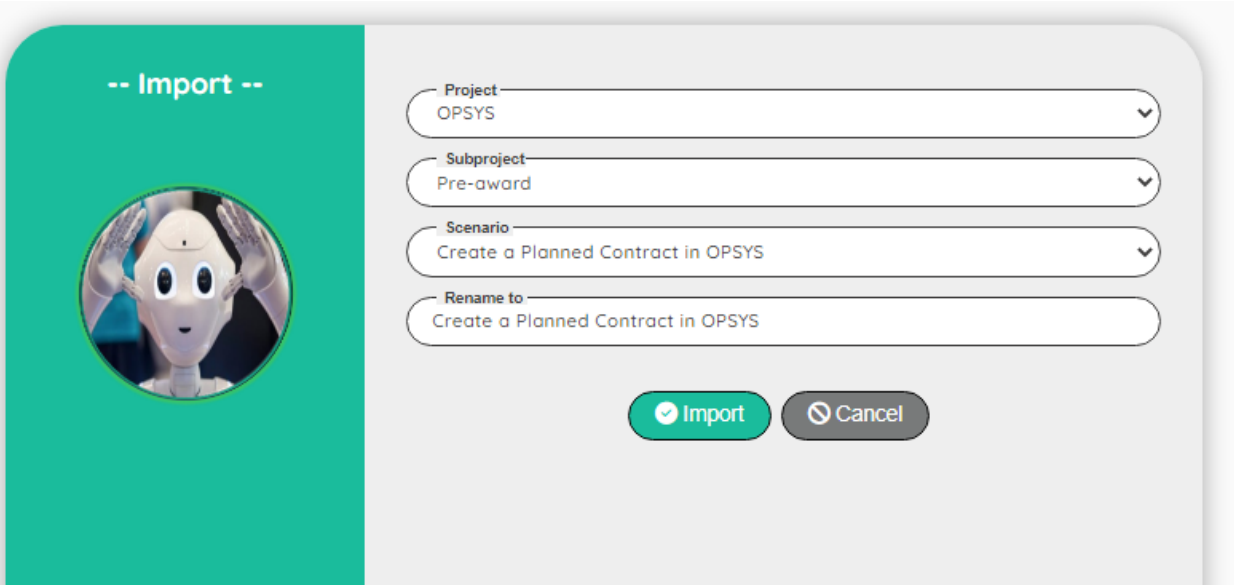
Yes

Save

Cancel

Topic	Icon	Comment
Subproject		For info: Name of the subproject
Scenario		Name of the scenario
Comment		Comment on the scenario
Active		Yes, No
Save		Save the edit
Cancel		Discard the edit

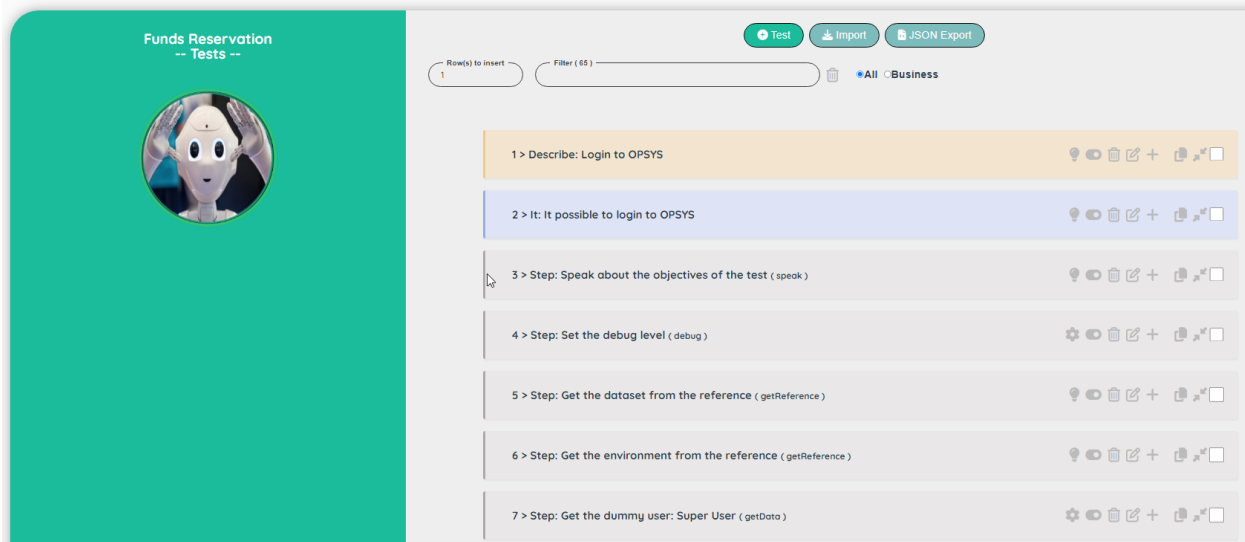
Scenario Import



Topic	Icon	Comment
Project		Select a project in the list
Subproject		Select a subproject in the list
Scenario		Select a scenario to import
Rename		You can update the name if necessary
Import		Import the scenario
Cancel		Discard the import

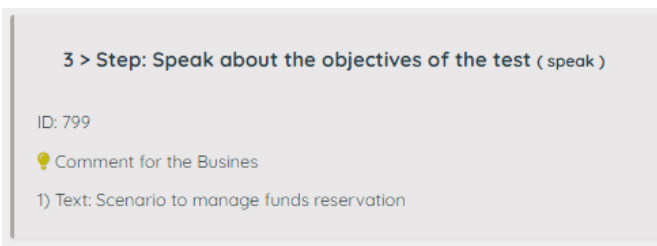
Test

The Test will allow you to manage the tests (steps)



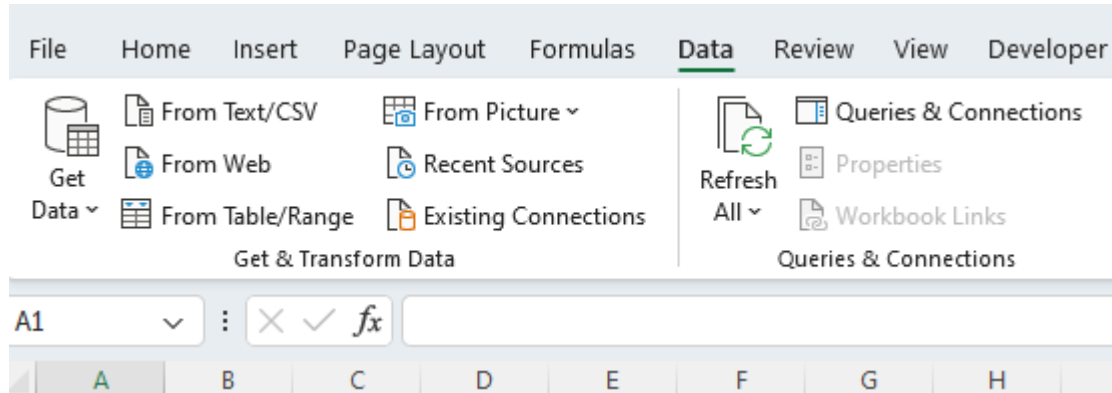
Topic	Icon	Comment
All / Business		Display all comments or only the ones for the Business
Add Test		Add a new test at the beginning of the list.
Active/Inactive		Set the test Inactive/Active
Business/Designer		Set the comment type to Business / Designer (Technical)
Cancel		Back to the Dashboard screen.
Import		Import a test(s) from another scenario (but in the same project)
JSON Export		Export the current test(s) into a .json file
Download	goffipl_test.json	Click on the link to download the .json file

Note: if you click on the comment, you will see extra information

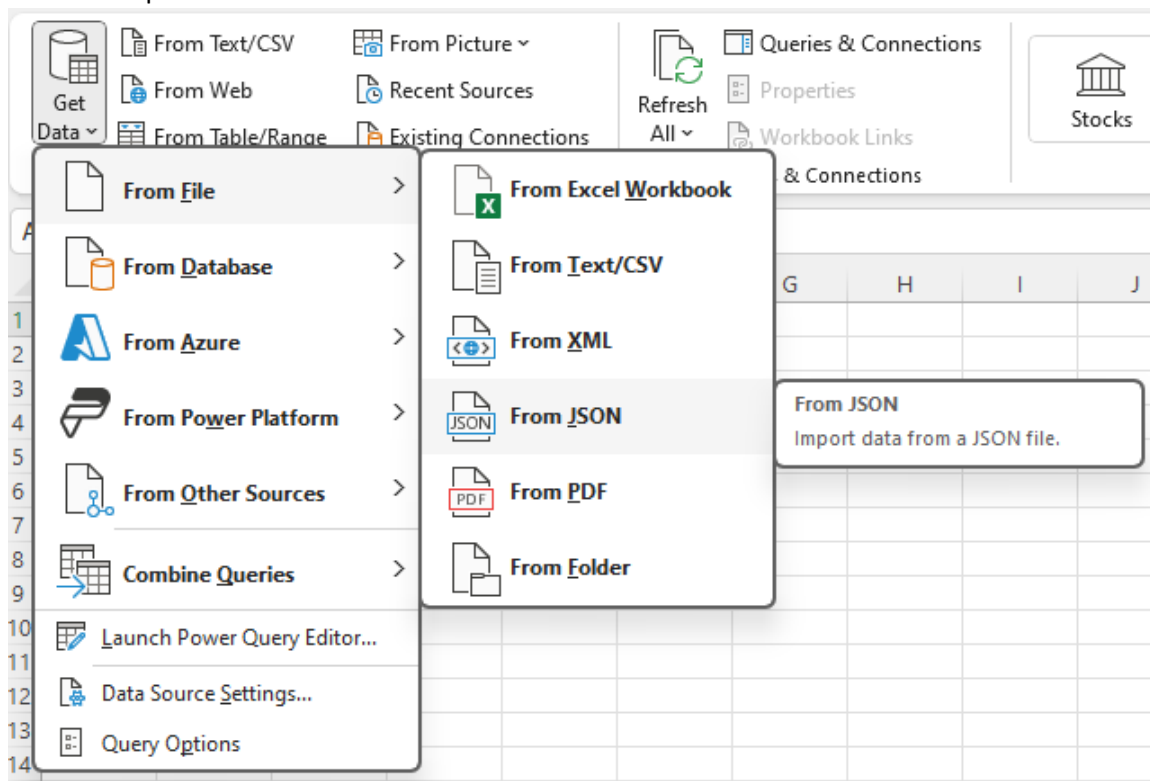


How to process the .json file into Excel

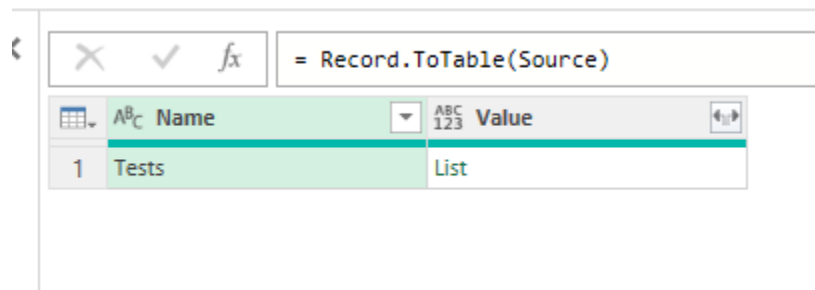
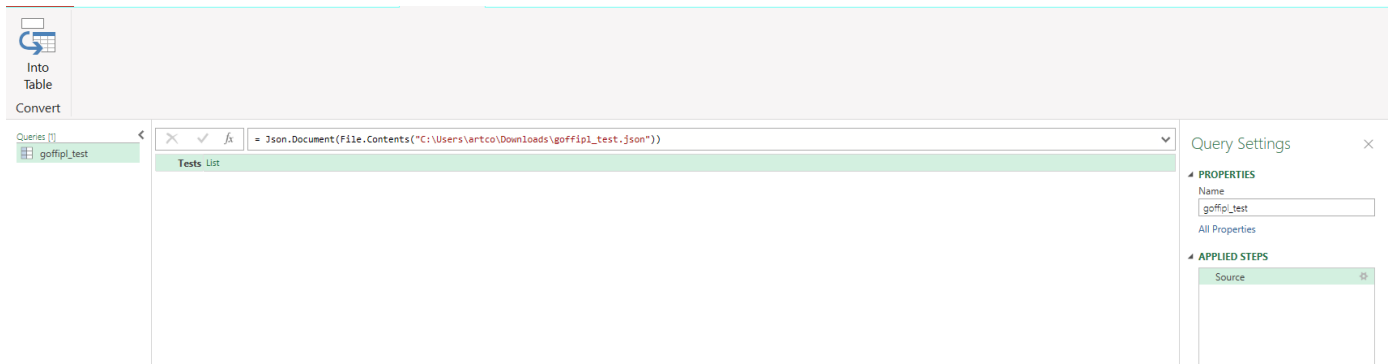
- Open a blank Excel workbook
- Go to the 'Data' section and click on 'Get Data'



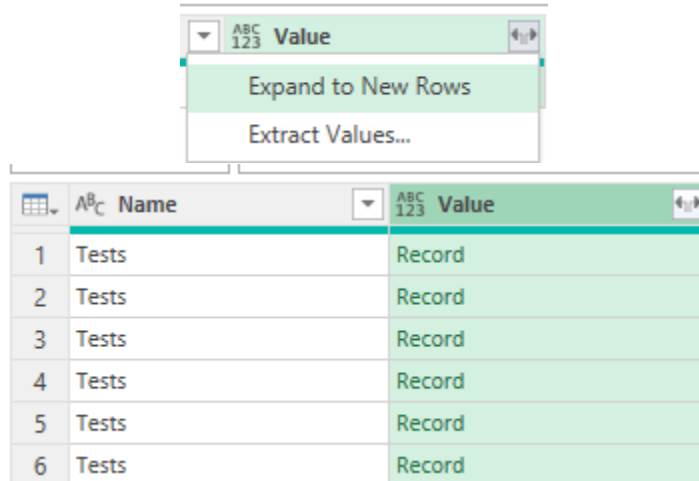
- Select the option 'From JSON'



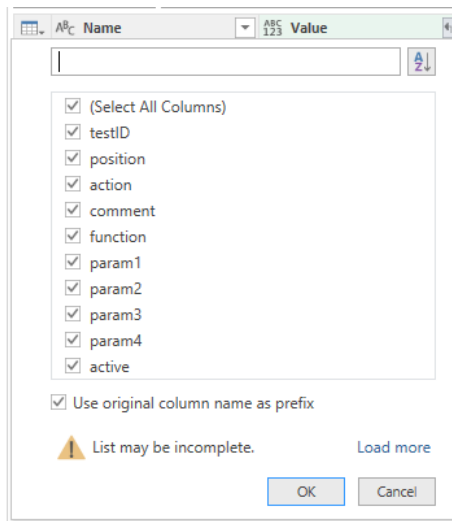
- Select the .json file from the folder 'Download' on your local driver.
a new window is now opened.
Click in the item of the menu 'Into Table Convert'



- Click on the right icon of the cell Value and select Expand to New rows



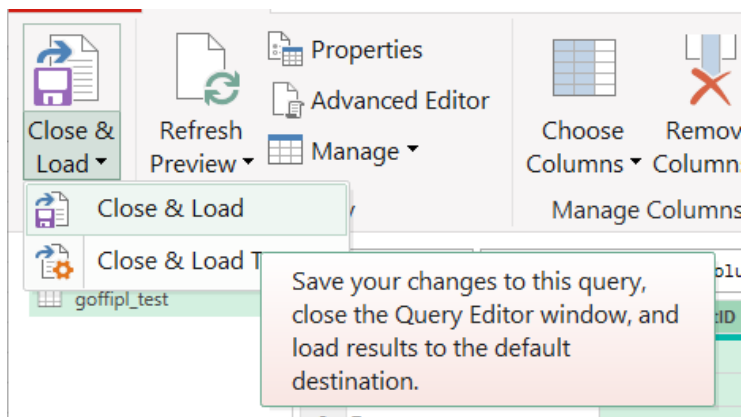
Click again on the right icon of the cell Value and click OK



The data are now displayed into a table.

Click the first item in the menu 'Close & Load'

	ABC Name	ABC Value.testID	ABC Value.position	ABC Value.action	ABC Value.comment
1	Tests	692	1	Describe	Setup Data
2	Tests	693	2	It	It possible to setup data (Database)
3	Tests	695	3	Step	Get the dataset
4	Tests	696	4	Step	Copy the Dataset into the Reference
5	Tests	697	5	Step	Get the environment
6	Tests	698	6	Step	Copy the Environment into the Reference



And now, you have a nice table with the tests of a scenario.

You can now manage this table to share with the Business Team.

For instance, you can remove the name of the function and the parameters and eventually, remove the technical test (like wait a little bit or wait for the refresh of the screen...)

	A	B	C	D	E	F	G
1	Name ▾	Value.testID ▾	Value.position ▾	Value.action ▾	Value.comment ▾	Value.function ▾	Value.param1
2	Tests	692	1	Describe	Setup Data	Not selected	
3	Tests	693	2	It	It possible to setup data (Dataset, Environment...)	Not selected	
4	Tests	695	3	Step	Get the dataset	getData	#Data_Dataset
5	Tests	696	4	Step	Copy the Dataset into the Reference	setReference	Dataset
6	Tests	697	5	Step	Get the environment	getData	#Data_Environme
7	Tests	698	6	Step	Copy the Environment into the Reference	setReference	Environment

Test Edit

Sanity Setup 1:
STAGING
-- Test --



Scenario

Sanity Setup 1: STAGING

Action

Step

Comment

Define the STAGING environment

Technical Comment

Condition

Function

setReference

Code

Sanity Environment

Value

STAGING

Reference Comment

Current environment for the sanity check

Status

Active

Save

Cancel

Topic	Icon	Comment
Scenario		For info: Name of the scenario
Comment	 Comment for the Business	Comment on the test
	 Comment for the Designer	
Technical		Text used for the natural language process
Condition		Any valid JavaScript expression. If the condition is false the test will be skipped!
Function	 / 	Select the name of the function (you can also filter by name or comment)
Parameters		Parameters depending of the selected function
Active		Yes, No
Save		Save the edit
Cancel		Discard the edit

Note: The comment can be defined as a text for the Business or for the Designer (more technical).

This is a simple way to indicate to the Business that want to review the scenario which steps are useful for him.

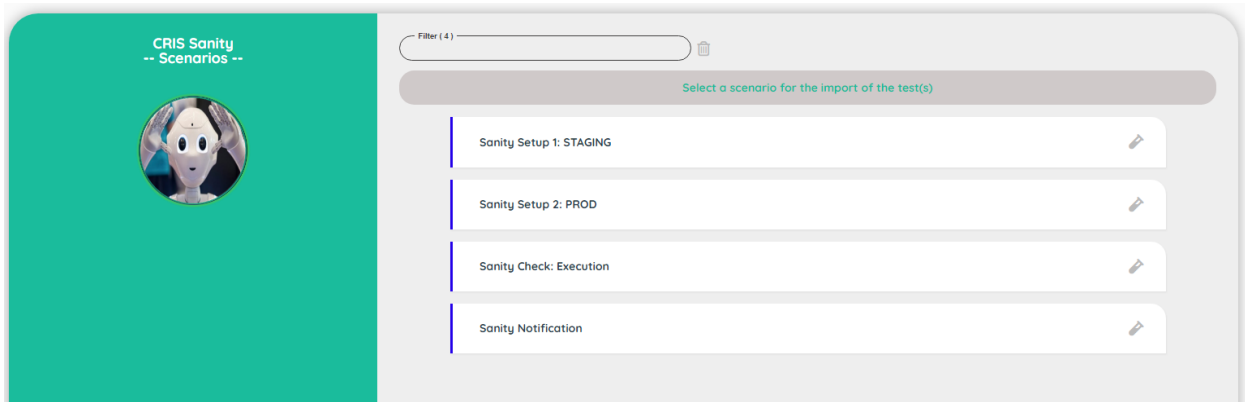
During the export, the field 'commentType' will be exported with the values:

- 1: Comment for the Business
- 0: Comment for the Designer (technical)

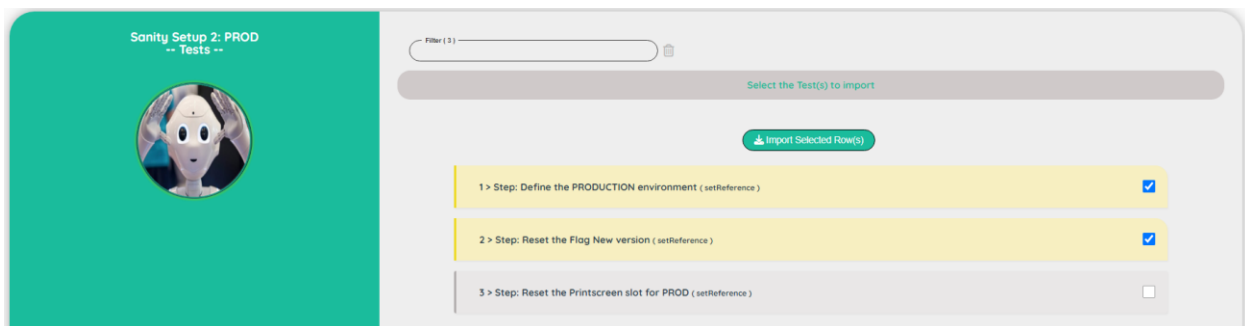
Import Test

The Import Test will allow you to import a test from another scenario (but from the same project).

First select a scenario



Select the test(s) that you want to import



Click on the button

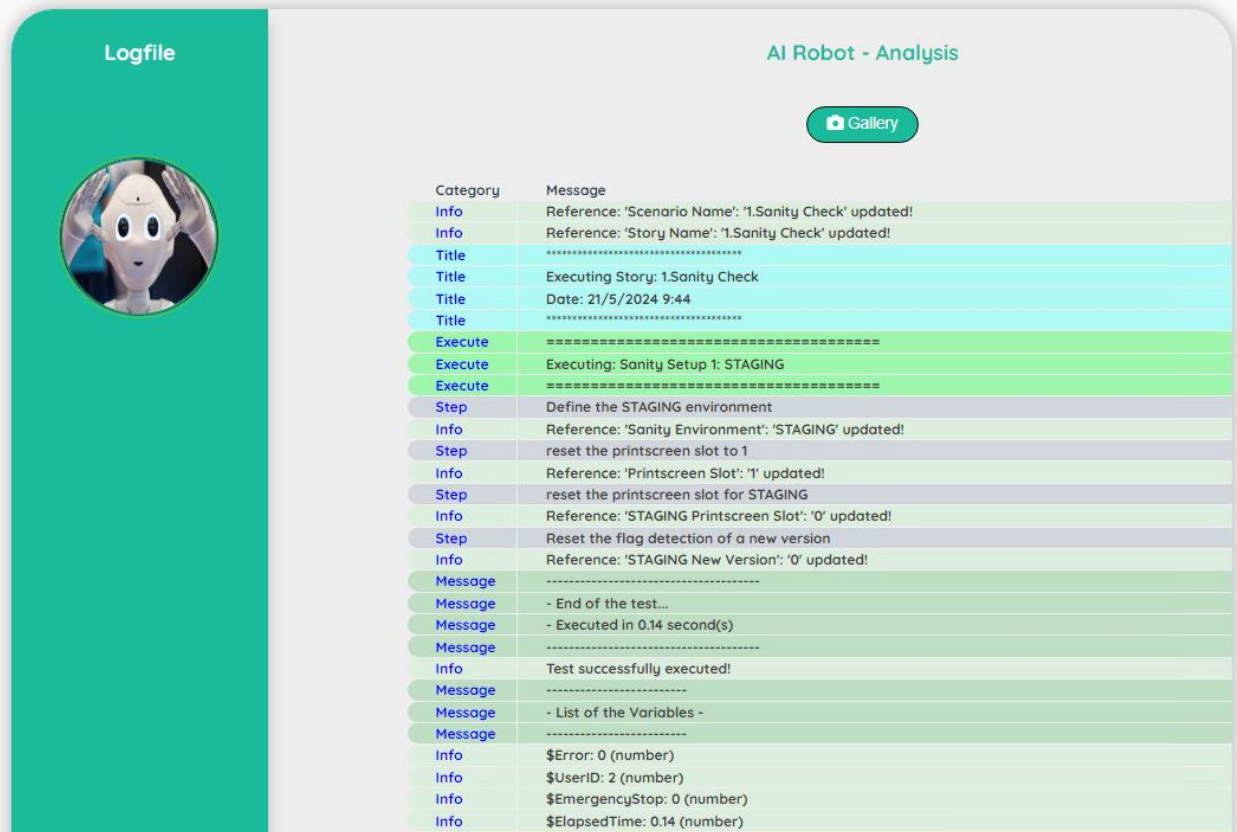


Logfile

The logfile will show you all the steps executed with extra information.

It helps you to understand how the test has been executed.

Note: There is one logfile by user. You see always the logfile of the last execution



Logfile

AI Robot - Analysis

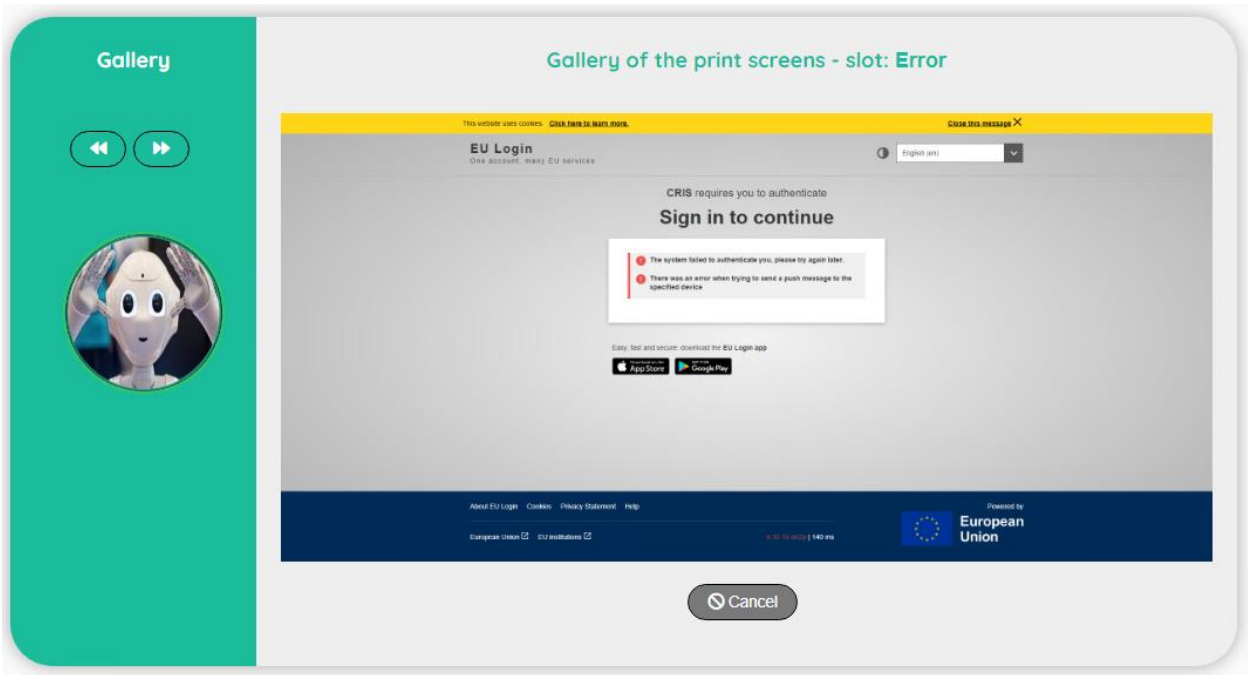
[Gallery](#)

Category	Message
Info	Reference: 'Scenario Name': '1.Sanity Check' updated!
Info	Reference: 'Story Name': '1.Sanity Check' updated!
Title	*****
Title	Executing Story: 1.Sanity Check
Title	Date: 21/5/2024 9:44
Title	*****
Execute	*****
Execute	Executing: Sanity Setup 1: STAGING
Execute	*****
Step	Define the STAGING environment
Info	Reference: 'Sanity Environment': 'STAGING' updated!
Step	reset the printscreen slot to 1
Info	Reference: 'Printscreen Slot': '1' updated!
Step	reset the printscreen slot for STAGING
Info	Reference: 'STAGING Printscreen Slot': '0' updated!
Step	Reset the flag detection of a new version
Info	Reference: 'STAGING New Version': '0' updated!
Message	*****
Message	- End of the test...
Message	- Executed in 0.14 second(s)
Message	*****
Info	Test successfully executed!
Message	*****
Message	- List of the Variables -
Message	*****
Info	\$Error: 0 (number)
Info	\$UserID: 2 (number)
Info	\$EmergencyStop: 0 (number)
Info	\$ElapsedTime: 0.14 (number)

Topic	Icon	Comment
Gallery		Access to the print screen(s)

Gallery

The gallery will show you the print screen taken during the execution of the story.
The first print screen is done automatically by the Robot when an error occurs (it will show you the last error detected – in case of success of a story, you will see the last error detected)

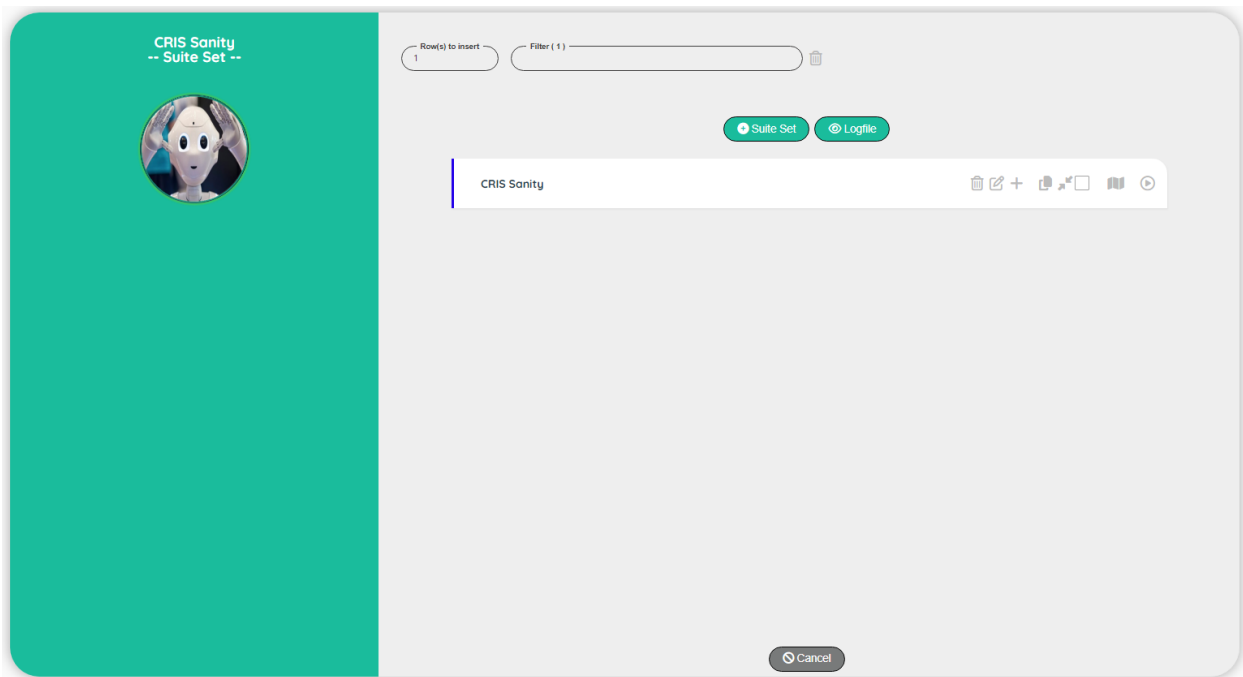


Topic	Icon	Comment
Previous		Go to the previous print screen
Next		Go to the next print screen
Cancel		Back to the Logfile

Suite Set

User Interface

The Suite will allow you to group different scenarios (you can also consult the best practices guide for more information about the suite). A suite is composed from a suite set (parent) and suites (children)



Topic	Icon	Comment
Suite		Go to the screen suite
Execute		Execute a suite.
Add Suite set		Add a new Suite at the beginning of the list.
Cancel		Back to the Dashboard screen.
Logfile		Go to the screen to view the Log file.

Suite Set Edit

CRIS Sanity

-- Suite set --



Created By

goffipl on: 25/03/2024

Updated By

goffipl on: 25/03/2024

Subproject

CRIS Sanity

Suite set

CRIS Sanity

Comment

Perform a sanity check on CRIS

Status

Active

Save

Cancel

Topic	Icon	Comment
Subproject		For info: Name of the subproject
Suite set		Name of the suite set
Comment		Comment for the suite set
Active		Yes, No
Save		Save the edit
Cancel		Discard the edit

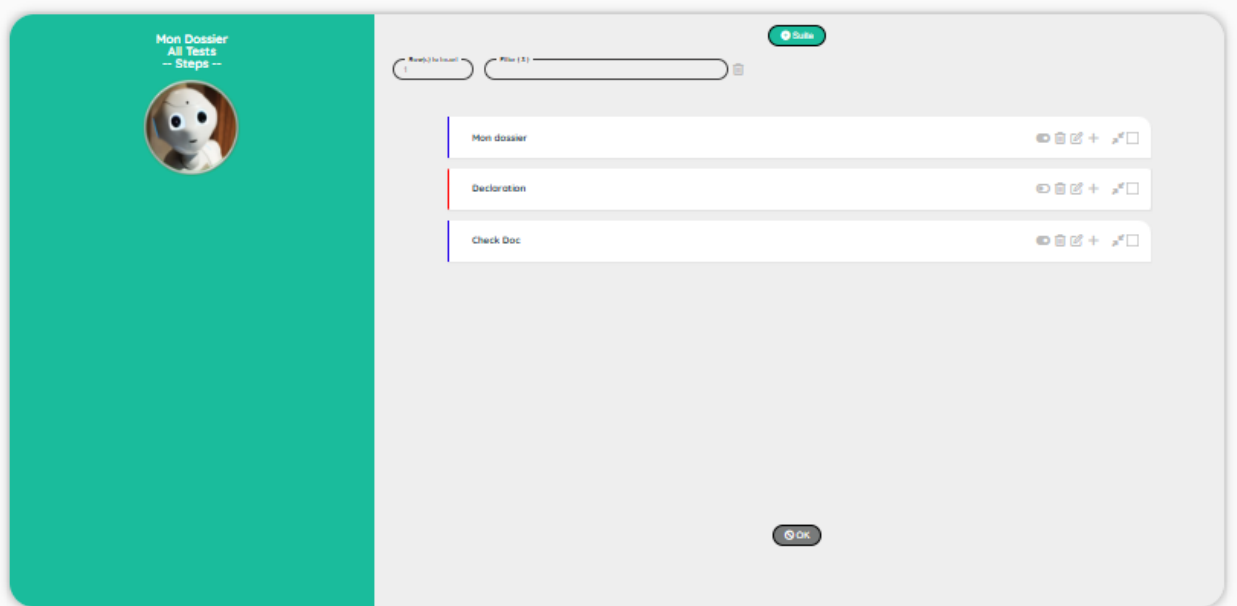
V1.12 – 13/04/2026

@ArtComputer 2026

Suites

User Interface

The Suites contains all the scenarios that you want to group, in order to execute multiple scenarios with one request



Topic	Icon	Comment
Add a Suite		Add a scenario at the beginning
Enable/Disable		Enable or Disable a suite.
Edit		Delete/Update/Insert/Move/Select.
OK		Back to the screen suite

Suites Set Edit

RRN Project

-- Suite --



Subproject

Mon Dossier

Suite

All Tests

Scenario

255

Comment

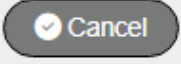
Mon dossier

Status

Active

Save

Cancel

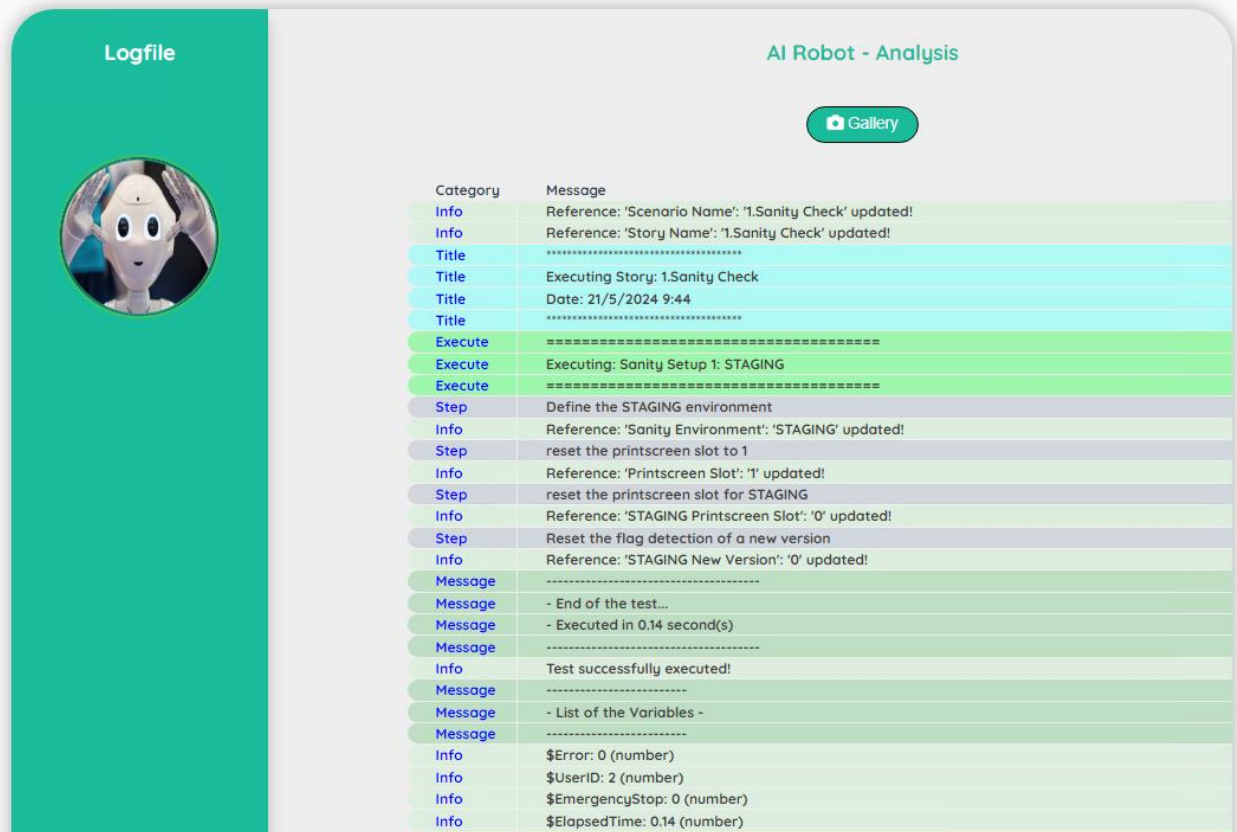
Topic	Icon	Comment
Select scenario		Select a scenario
Comment		Comment for the suites step
Active		Yes, No
Save		Save the edit
Cancel		Discard the edit

Logfile

The logfile will show you all the steps executed with extra information.

It helps you to understand how the test has been executed.

Note: There is one logfile by user. You see always the logfile of the last execution



Logfile

AI Robot - Analysis

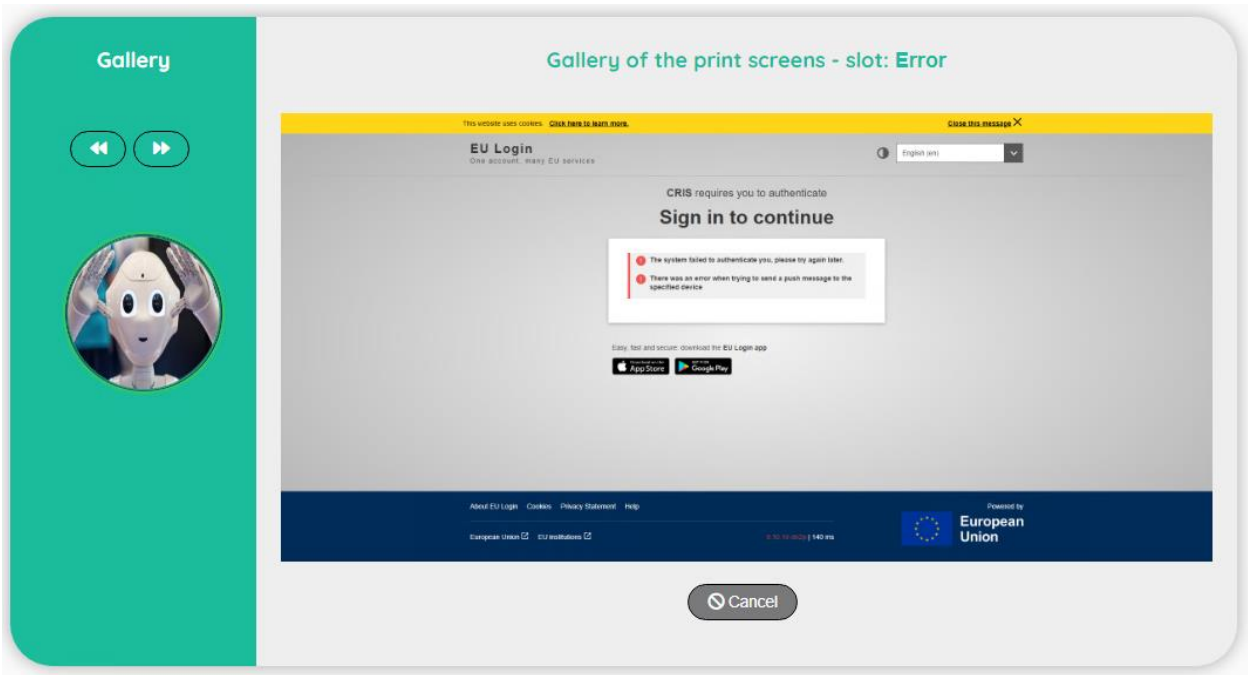
[Gallery](#)

Category	Message
Info	Reference: 'Scenario Name': '1.Sanity Check' updated!
Info	Reference: 'Story Name': '1.Sanity Check' updated!
Title	*****
Title	Executing Story: 1.Sanity Check
Title	Date: 21/5/2024 9:44
Title	*****
Execute	*****
Execute	Executing: Sanity Setup 1: STAGING
Execute	*****
Step	Define the STAGING environment
Info	Reference: 'Sanity Environment': 'STAGING' updated!
Step	reset the printscreen slot to 1
Info	Reference: 'Printscreen Slot': '1' updated!
Step	reset the printscreen slot for STAGING
Info	Reference: 'STAGING Printscreen Slot': '0' updated!
Step	Reset the flag detection of a new version
Info	Reference: 'STAGING New Version': '0' updated!
Message	*****
Message	- End of the test...
Message	- Executed in 0.14 second(s)
Message	*****
Info	Test successfully executed!
Message	*****
Message	- List of the Variables -
Message	*****
Info	\$Error: 0 (number)
Info	\$UserID: 2 (number)
Info	\$EmergencyStop: 0 (number)
Info	\$ElapsedTime: 0.14 (number)

Topic	Icon	Comment
Gallery		Access to the print screen(s)

Gallery

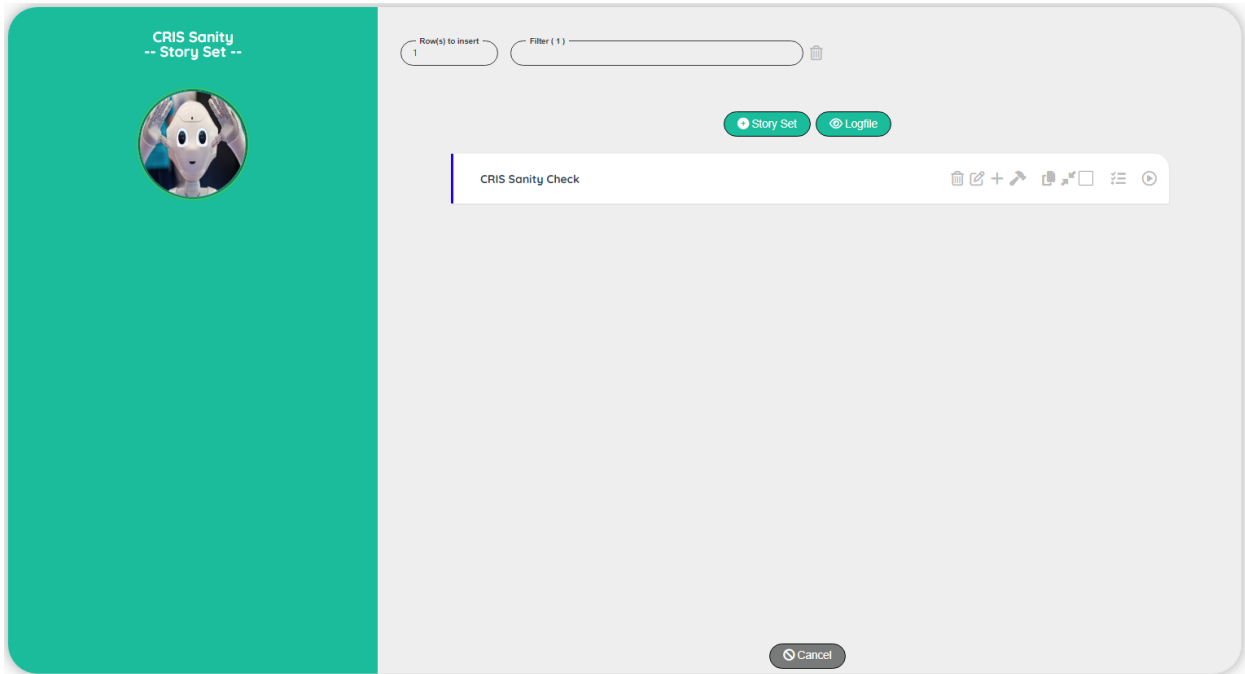
The gallery will show you the print screen taken during the execution of the story.
The first print screen is done automatically by the Robot when an error occurs (it will show you the last error detected – in case of success of a story, you will see the last error detected)



Topic	Icon	Comment
Previous		Go to the previous print screen
Next		Go to the next print screen
Cancel		Back to the Logfile

Story Set

Story set



Topic	Icon	Comment
Stories		Go to the screen Stories.
Execute		Execute a Story.
Parameters		Add a story at the beginning of the list
Add Story		Add a story at the beginning of the list
Cancel		Back to the control panel
Logfile		Go to the screen to view the Log file.

Story set Edit

CRIS Sanity

-- Story set --



Created By

goffipl on: 27/03/2024

Updated By

goffipl on: 27/03/2024

Subproject

CRIS Sanity

Story set

CRIS Sanity Check

Comment

CRIS Sanity Check

Status

Active (restricted to Designer & Admin)

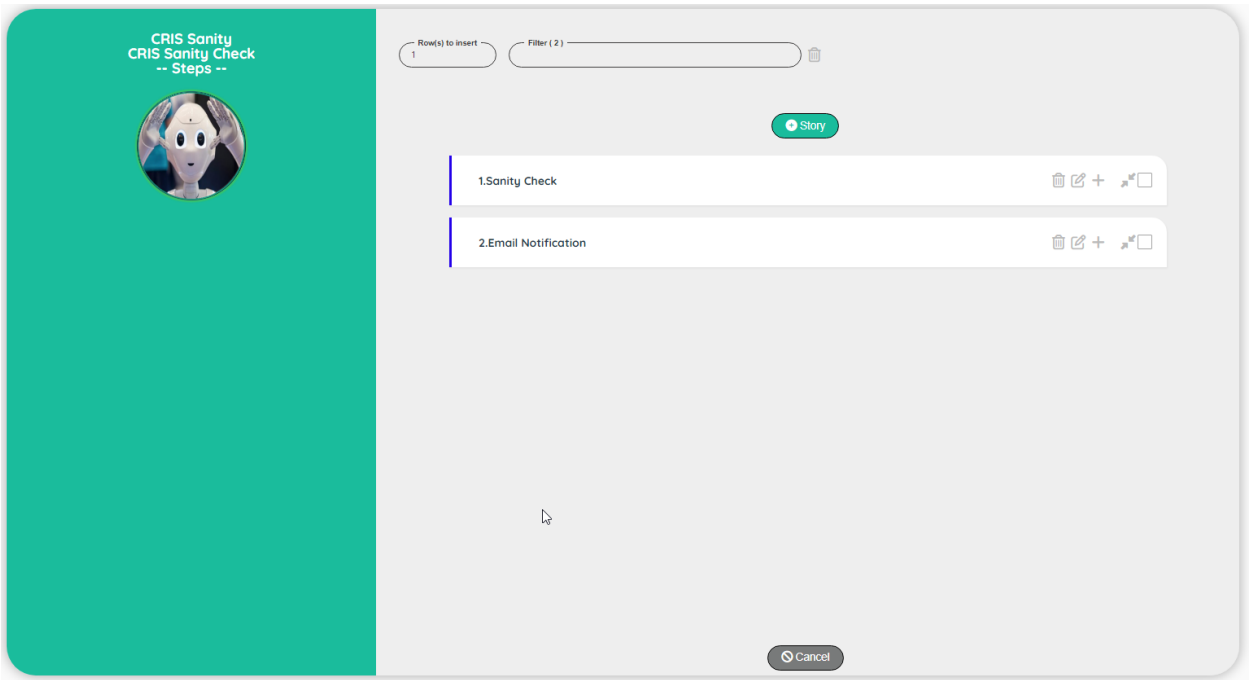
Save

Cancel

Topic	Icon	Comment
Subproject		For info: Name of the subproject
Story set		Name of the story set
Comment		Comment for the story set
Active		Active, Not active, Publish
Save		Save the edit
Cancel		Discard the edit

Note: The Tester can see only the Published story (Active and Published are visible by the Designer and the Administrator)

Stories



Topic	Icon	Comment
Add Story		Add a story at the beginning of the list
Cancel		Back to the Story set

Stories Edit

CRIS
-- Story --



Subproject
CRIS Sanity

Story set
CRIS Sanity Check

Story
1.Sanity Check

Graph label
Sanity Check

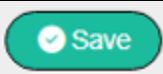
Selector
Suite

Suite
10

Comment
Perform a sanity check on CRIS

Status
Active

Save Cancel

Topic	Icon	Comment
Subproject		For info: Name of the subproject
Story set		For info: Name of the story set
Story		Name of the story
Graph label		Scenario or Suite
Selector		Comment for the story set
Selector Info		ID and Comment on the selected selector
Active		Active, Not active, Publish
Save		Save the edit
Cancel		Discard the edit

Logfile

The logfile will show you all the steps executed with extra information.

It helps you to understand how the test has been executed.

Note: There is one logfile by user. You see always the logfile of the last execution

Logfile

AI Robot - Analysis

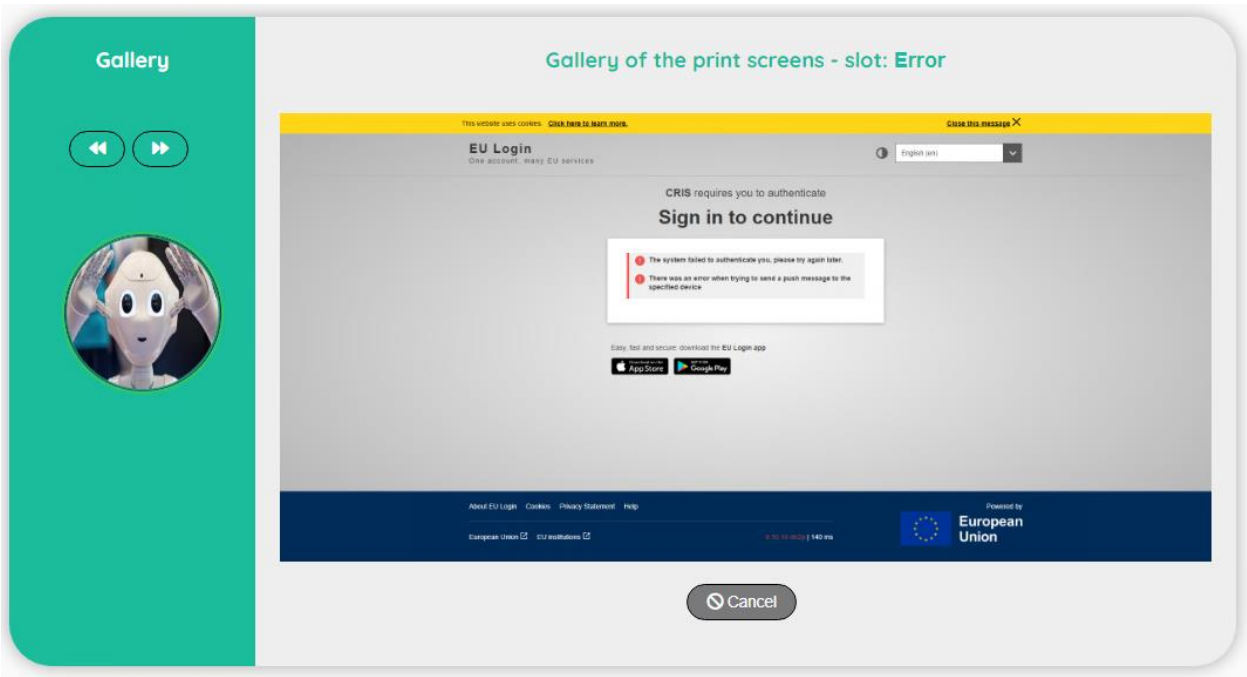
[Gallery](#)

Category	Message
Info	Reference: 'Scenario Name': '1.Sanity Check' updated!
Info	Reference: 'Story Name': '1.Sanity Check' updated!
Title	*****
Title	Executing Story: 1.Sanity Check
Title	Date: 21/5/2024 9:44
Title	*****
Execute	*****
Execute	Executing: Sanity Setup 1: STAGING
Execute	*****
Step	Define the STAGING environment
Info	Reference: 'Sanity Environment': 'STAGING' updated!
Step	reset the printscreen slot to 1
Info	Reference: 'Printscreen Slot': '1' updated!
Step	reset the printscreen slot for STAGING
Info	Reference: 'STAGING Printscreen Slot': '0' updated!
Step	Reset the flag detection of a new version
Info	Reference: 'STAGING New Version': '0' updated!
Message	*****
Message	- End of the test...
Message	- Executed in 0.14 second(s)
Message	*****
Info	Test successfully executed!
Message	*****
Message	- List of the Variables -
Message	*****
Info	\$Error: 0 (number)
Info	\$UserID: 2 (number)
Info	\$EmergencyStop: 0 (number)
Info	\$ElapsedTime: 0.14 (number)

Topic	Icon	Comment
Gallery		Access to the print screen(s)

Gallery

The gallery will show you the print screen taken during the execution of the story.
The first print screen is done automatically by the Robot when an error occurs (it will show you the last error detected – in case of success of a story, you will see the last error detected)

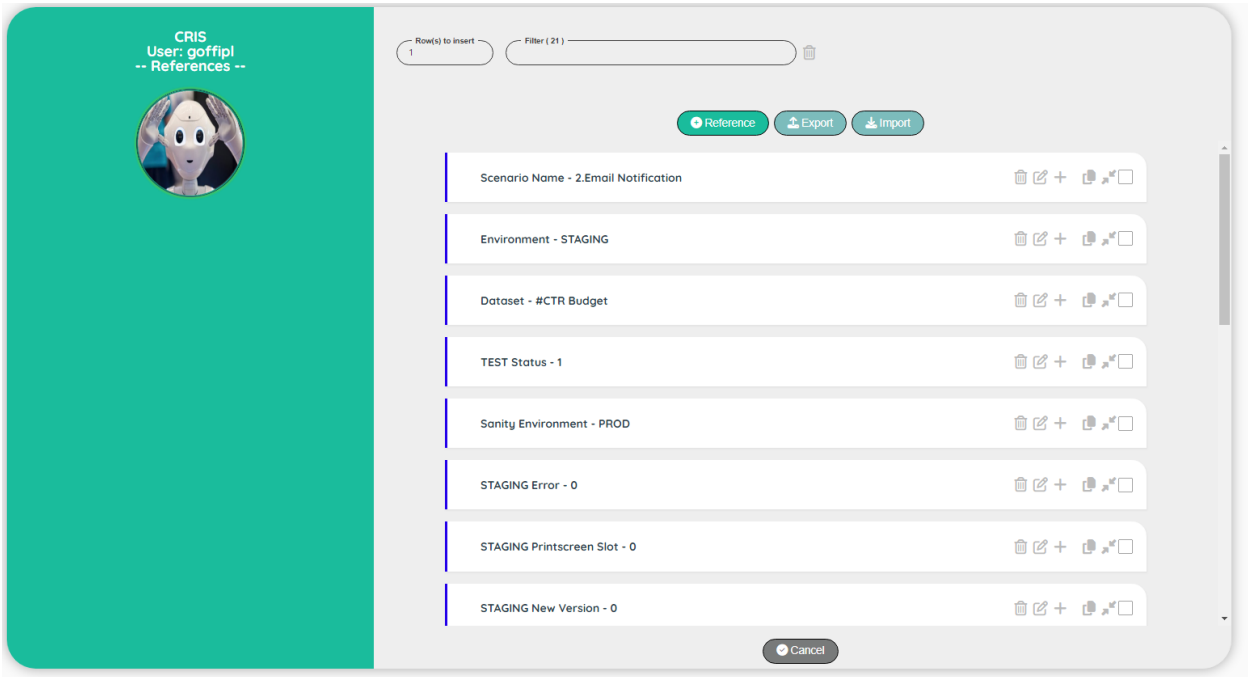


Topic	Icon	Comment
Previous		Go to the previous print screen
Next		Go to the next print screen
Cancel		Back to the Logfile

Reference

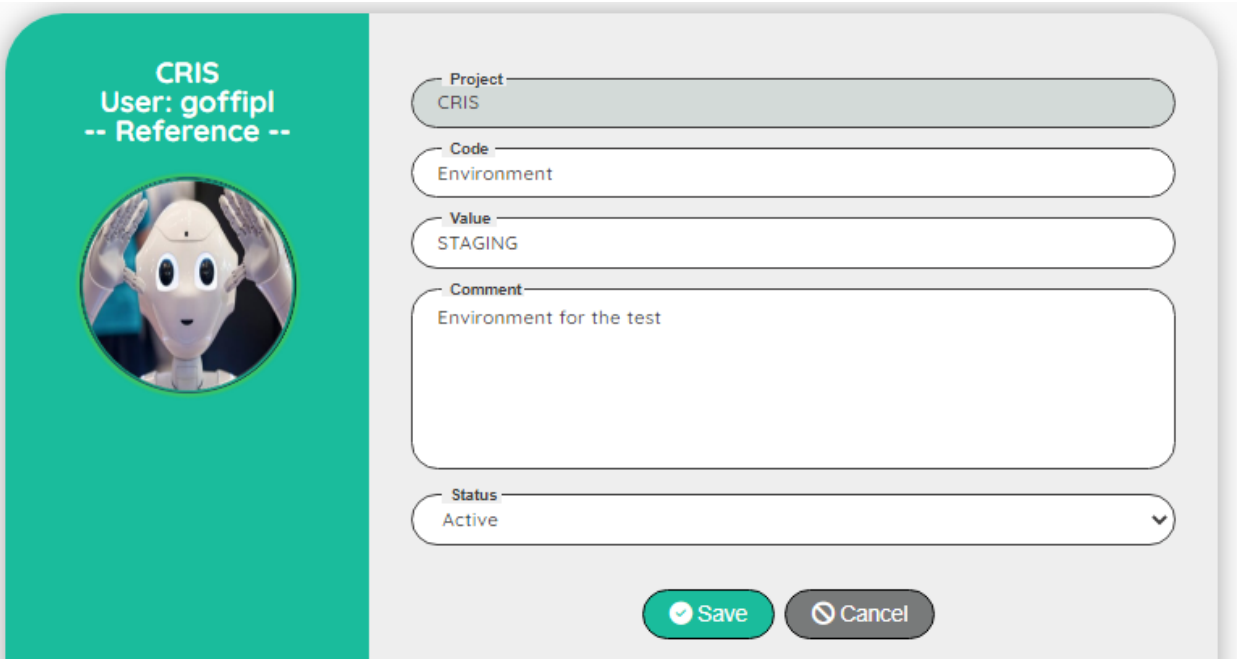
Reference

The reference will show all the values stored by the scenarios in order to exchange data.
A reference is own by a specific user for a project.

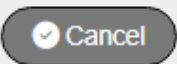


Topic	Icon	Comment
Add Reference		Add a reference at the beginning of the list.
Cancel		Back to the control panel.
Export		Go to the screen to export a reference.
Import		Go to the screen to import a reference.

Reference Edit



The image shows a 'Reference Edit' form. On the left, a teal vertical bar contains the text 'CRIS User: goffipl -- Reference --' and a circular profile picture of a robot head. The main form area on the right has a light gray background and contains several input fields: 'Project' (CRIS), 'Code' (Environment), 'Value' (STAGING), 'Comment' (Environment for the test), and 'Status' (Active). At the bottom right are two buttons: a green 'Save' button and a gray 'Cancel' button.

Topic	Icon	Comment
Project		For info: Name of the project
Code		Code of the reference
Value		Value of the reference
Comment		Comment for the reference
Active		Active, Not active
Save		Save the edit
Cancel		Discard the edit

Note: The reference 'Environment' is very important for the tool, be sure to create one and keep the value relevant to your tests ('Environment' is used in the Dashboard to display the graph with the performances).

Example of value for Environment could be for instance: PROD, ACC, TEST, STAGING....

Export Reference

The export will allow you to take a backup or to share your reference with another colleague.

-- Export --


Project
OPSYS

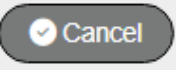
User
goffipl

Slot
Slot 1

Comment
Export for Diego: SEA-2023-24775

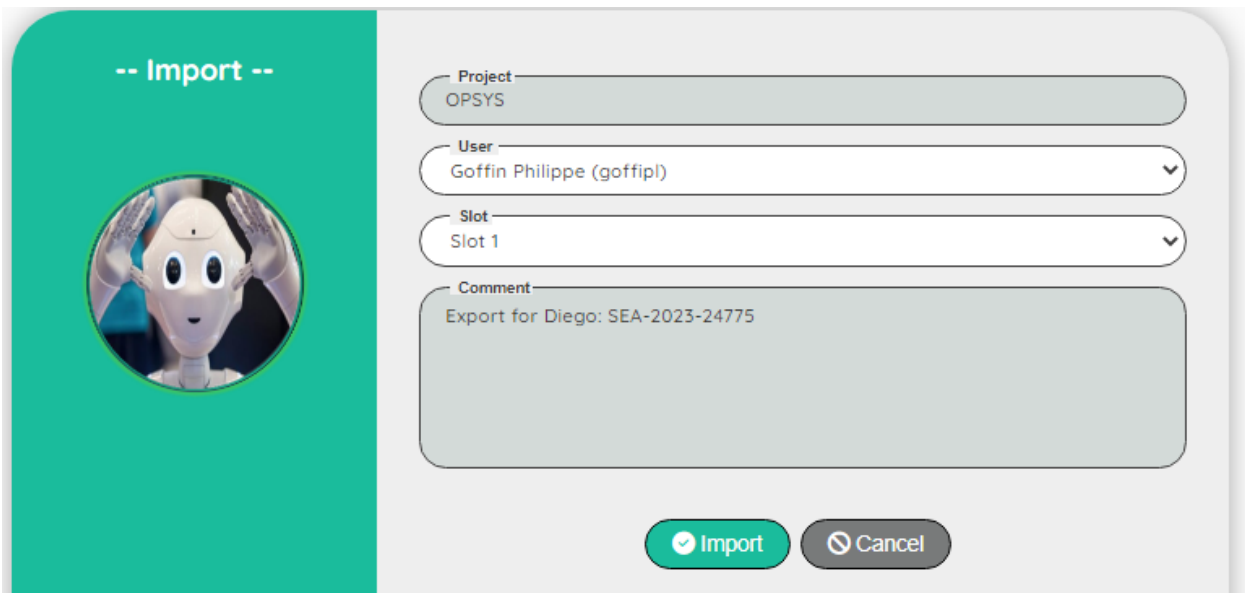
Export

Cancel

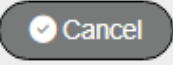
Topic	Icon	Comment
Project		For info: Name of the project
User		For info: Login of the user
Slot		10 slots are available to back up your references
Comment		Comment on the export
Export		Export the references
Cancel		Back to the control panel.

Import Reference

The import will allow you to restore a backup or get a reference from another colleague. Be careful, you current references will be erased and replaced by the import



The dialog box is titled "-- Import --" and features a circular profile picture of a robot on the left. On the right, there are four input fields: "Project" (text input with "OPSYS"), "User" (dropdown menu with "Goffin Philippe (goffipl)"), "Slot" (dropdown menu with "Slot 1"), and "Comment" (text area with "Export for Diego: SEA-2023-24775"). At the bottom right, there are two buttons: a green "Import" button with a checkmark icon and a grey "Cancel" button with a close icon.

Topic	Icon	Comment
Project		For info: Name of the project
User		Select a user in the list
Slot		Select a slot
Comment		For info: Comment on the export
Import		Import the references
Cancel		Back to the control panel.

Specific Reference

When you create a new project, it can be a good idea to define some specific reference to impact your tests.

None of the specific references are mandatory (a default value will be used).

In the scenario, you can also write a step to update a specific reference (for instance the default timeout in seconds used by playwright to detect an element).

Be careful, the name of the reference is case sensitive!

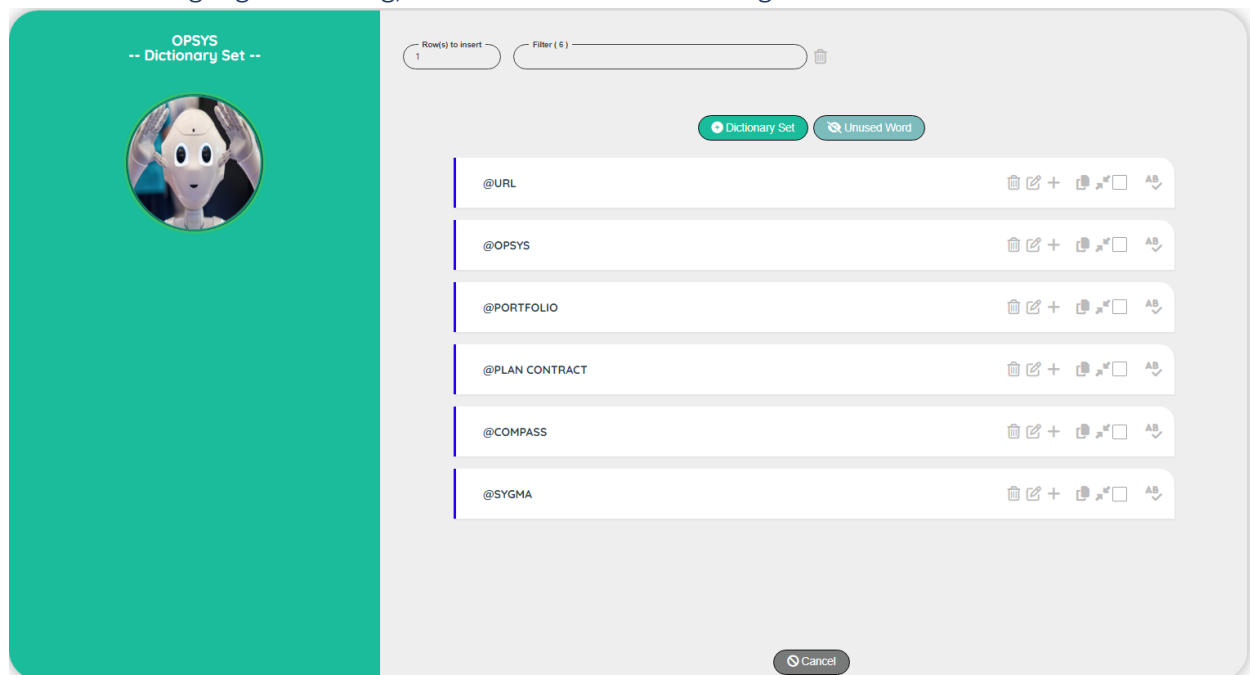
Reference	Default value	Comment
TimeOut	30	Playwright time out (in seconds) to detect an element
Headless	0	0: no headless browser (default), 1: headless browser
Browser	Chrome	Possible value: Chrome, Firefox or Safari
Device	<N/A>	A specific smartphone, a tablet or a desktop computer. Check the list of available devices on the playwright documentation. The device will impact the size of the browser!
Environment		Used by the performance graph (E.g.: ACC or PROD, INT....)
AllEnvironments		Used by the performance graph. A list of value comma separated (E.g.: ACC,PROD)

There are also technical references created by the tool during the execution of the tests.
These references are used by the tool to define context, to return the status of the execution, to restart at the right step after an error...

Dictionary

Dictionary set

The dictionary is available for a project. It is composed of a dictionary set and a dictionary word. You can store URL, xpath or even translation (there is a field language that you can use) For the translation, the logic is the following: if you ask for an unknown language, the record with the language '*' will be used. So, you can for instance, use '*' for English and if a translation in another language is missing, the user will receive the English translation.



Topic	Icon	Comment
Dictionary		Go to the screen dictionary word
Add Dictionary		Add a dictionary set at the beginning of the list.
Unused word		Go to the screen to export a reference.
Cancel		Back to the control panel.

Dictionary set Edit

OPSYS

-- Dictionary Set --



Project
OPSYS

Code
@URL

Comment

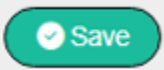
All the links to the applications (in all the environments)

Status

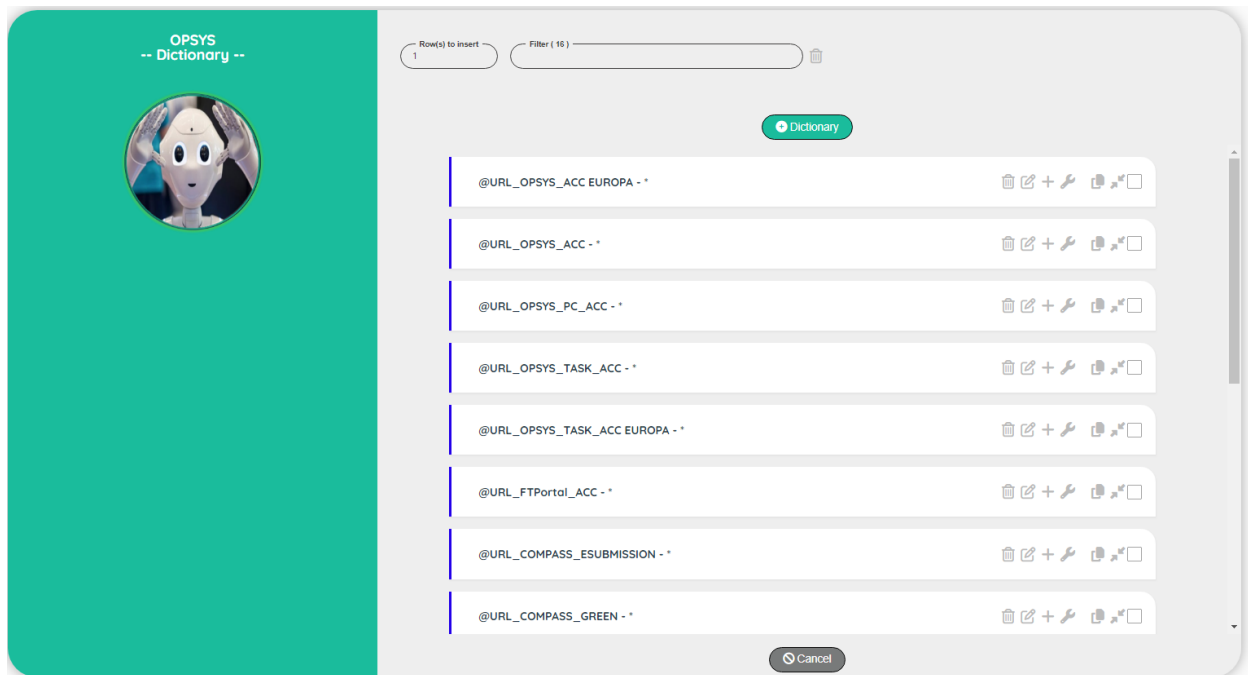
Active

Save

Cancel

Topic	Icon	Comment
Project		For info: Name of the project
Code		Code of the dictionary must start by @
Comment		Comment for the dictionary
Active		Active, Not active
Save		Save the edit
Cancel		Discard the edit

Dictionary word



Topic	Icon	Comment
Add Dictionary		Add a word at the beginning of the list.
Cancel		Back to the control panel.

Dictionary word Edit

Demo Tosca
 -- Dictionary --
 

Created By
 goffipl on: 23/01/2026

Updated By
 goffipl on: 23/01/2026

 Scanning

Project
 Demo Tosca

Header Code
 @SignInPortal

Code
 _Username

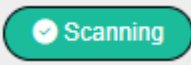
Language
 *

Value
 //form[@id= 'second_form']/div/input[@id='usr']

Comment
 Field Username

Status
 Active

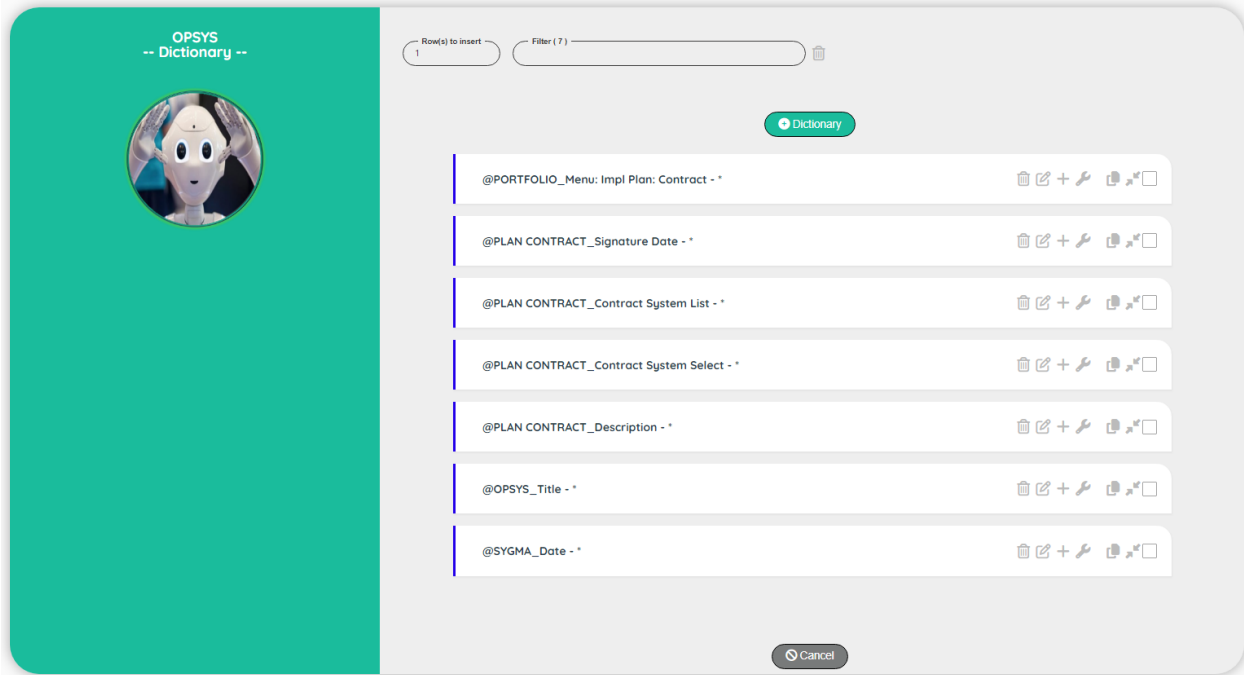
 Save
  Cancel

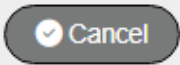
Topic	Icon	Comment
Project		For info: Name of the project
Dictionary set		For info: Name of the dictionary set
Code		Code of the dictionary must start by @
Language		* by default, but you can use an ISO code (EN, FR...)
Comment		Comment for the dictionary word
Active		Active, Not active
Save		Save the edit
Cancel		Discard the edit
Scanning		Scan a web page to detect elements

Dictionary unused word

This screen will show you all the unused word(s).

Be careful, the detection can be fooled when you use a variable to replace a partial name of a word. For instance if you have @URL_Acceptance, it will be flagged as unused if you use the following step in your scenario: @URL_Environment.

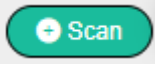


Topic	Icon	Comment
Cancel		Discard the edit

Dictionary scan web element

This screen will allow you to scan a web page to identify a specific web element. From the Dictionary Edit screen, click on the button Scanning.

The screenshot shows a web application interface for scanning web elements. On the left, a teal sidebar contains the text "Demo Tosca -- Scanning --" and a circular icon of a robot. The main content area is light gray. At the top, there is a "Scan" button with a magnifying glass icon. Below it is a "URL" input field containing the text "https://anupdamoda.github.io/AceOnlineShoePo". Underneath the URL field are seven output fields, each with a label and a value: "tagName" (N/A), "id" (N/A), "name" (N/A), "class" (N/A), "innerText" (N/A), "placeholder" (N/A), and "xpath" (N/A). At the bottom of the main area are two buttons: "Submit" and "Cancel".

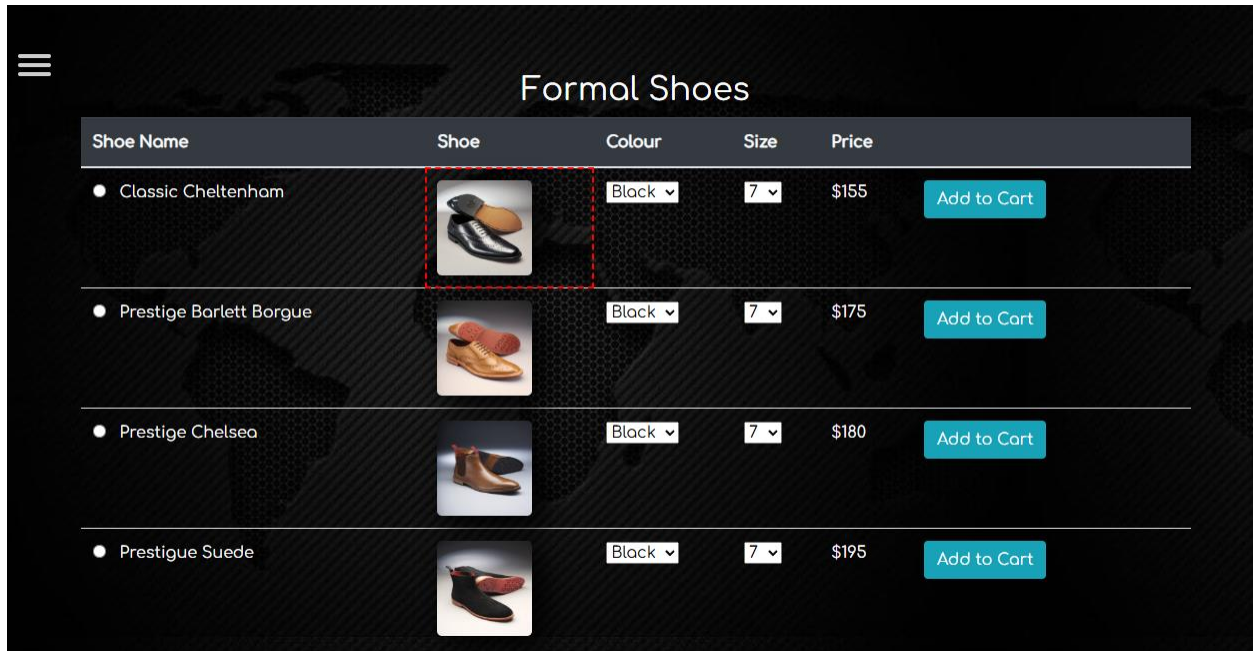
Topic	Icon	Comment
URL		The URL of the web page you want to scan
Scan		Scan a web page
Device		Web browser or Phone browser
Delay		Delay (in sec) before scanning elements

For the example, I used the link:

<https://anupdamoda.github.io/AceOnlineShoePortal/FormalShoeslist.html>

Use the mouse to select an element – hover the mouse on the element and a red dashed box will be displayed (for instance the picture of the shoes)

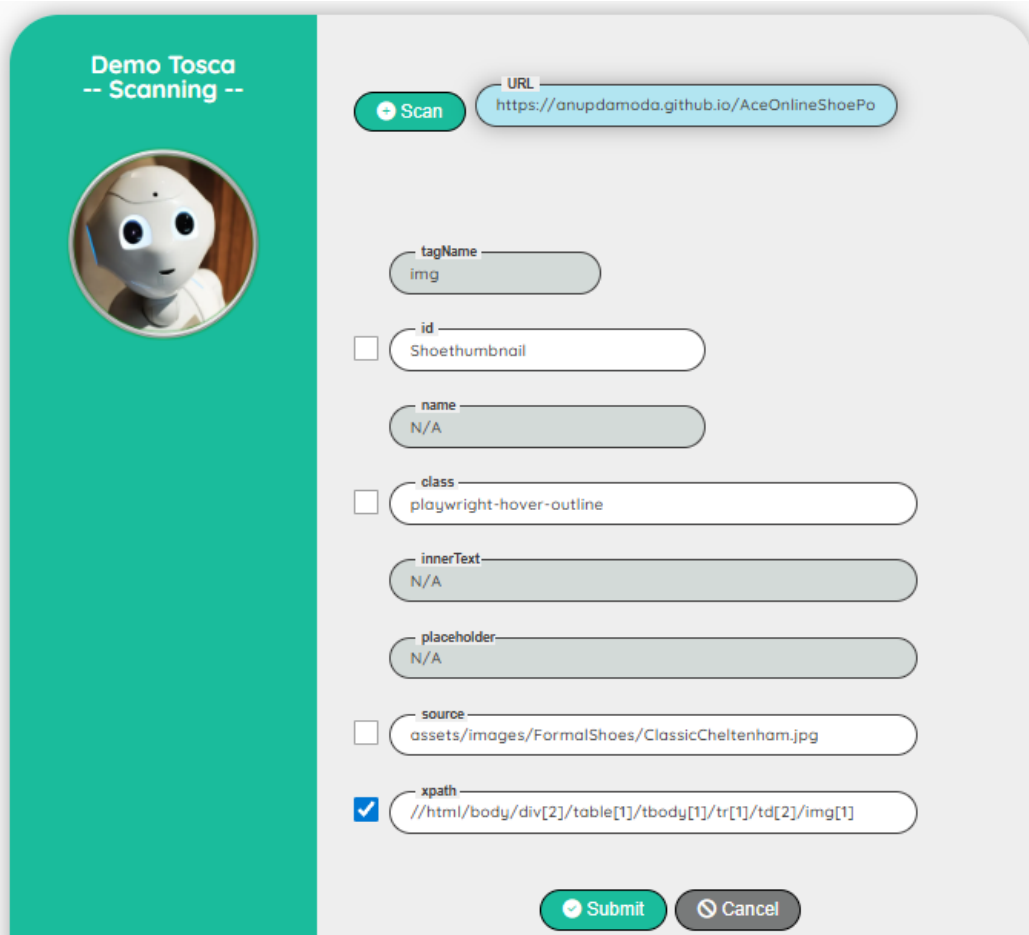
When you are satisfied, click with the mouse. The browser will be closed and you will back to the Robot.



Note 1: if the browser is not closed after the click, just close the external window.

Note 2: if you want to scan the Phone browser, you need first to launch Android Studio and start a phone with the Device Manager.

Depending of the element (image, button...) some attributes will be displayed or not.



The screenshot shows the 'Demo Tosca -- Scanning --' interface. On the left, there is a teal sidebar with a robot icon. The main area is light gray and contains a 'Scan' button and a 'URL' field with the value 'https://anupdamoda.github.io/AceOnlineShoePo'. Below these are several attribute fields, each with a checkbox to its left:

- tagName**: img
- id**: Shoethumbnail (checkbox is unchecked)
- name**: N/A
- class**: playwright-hover-outline (checkbox is unchecked)
- innerText**: N/A
- placeholder**: N/A
- source**: assets/images/FormalShoes/ClassicCheltenham.jpg (checkbox is unchecked)
- xpath**: //html/body/div[2]/table[1]/tbody[1]/tr[1]/td[2]/img[1] (checkbox is checked)

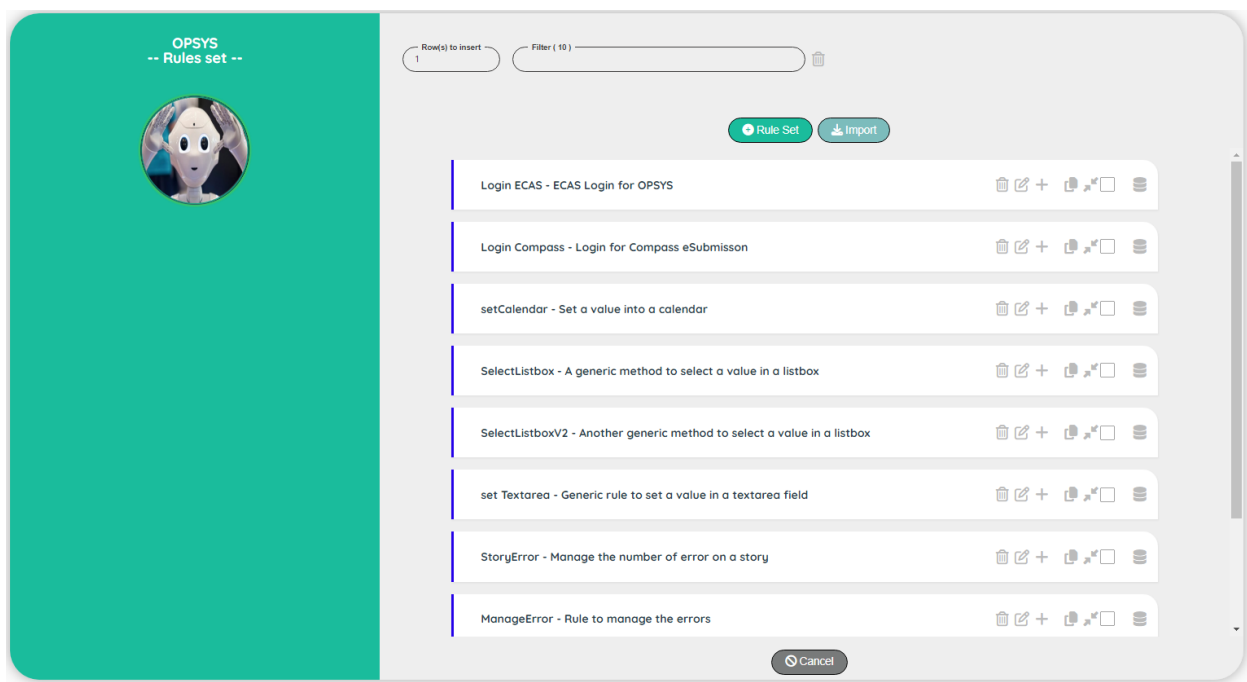
At the bottom right, there are 'Submit' and 'Cancel' buttons.

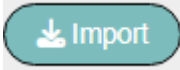
In this example, the robot was able to detect the Id, the class, the source and the xpath (selected by default). By clicking on the button Submit, the dictionary will be updated automatically!

Rules

Rules set

The rules are available for a project. It is composed of a rule set and rules. The rules can be used by the robot to make a decision or by the Designer to define subroutine that can be reused. All the functions are available also in the rules.



Topic	Icon	Comment
Rules		Go to the screen Rules
Add Rule set		Add a rule set at the beginning of the list.
Import rules		Import rules from another project
Cancel		Back to the control panel.

Rules set Edit

OPSYS

-- Rule set --



Created By

goffipl on: 01/03/2024

Updated By

goffipl on: 26/04/2024

Project

OPSYS

Rule set

Login ECAS

Comment

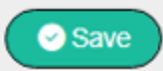
ECAS Login for OPSYS

Status

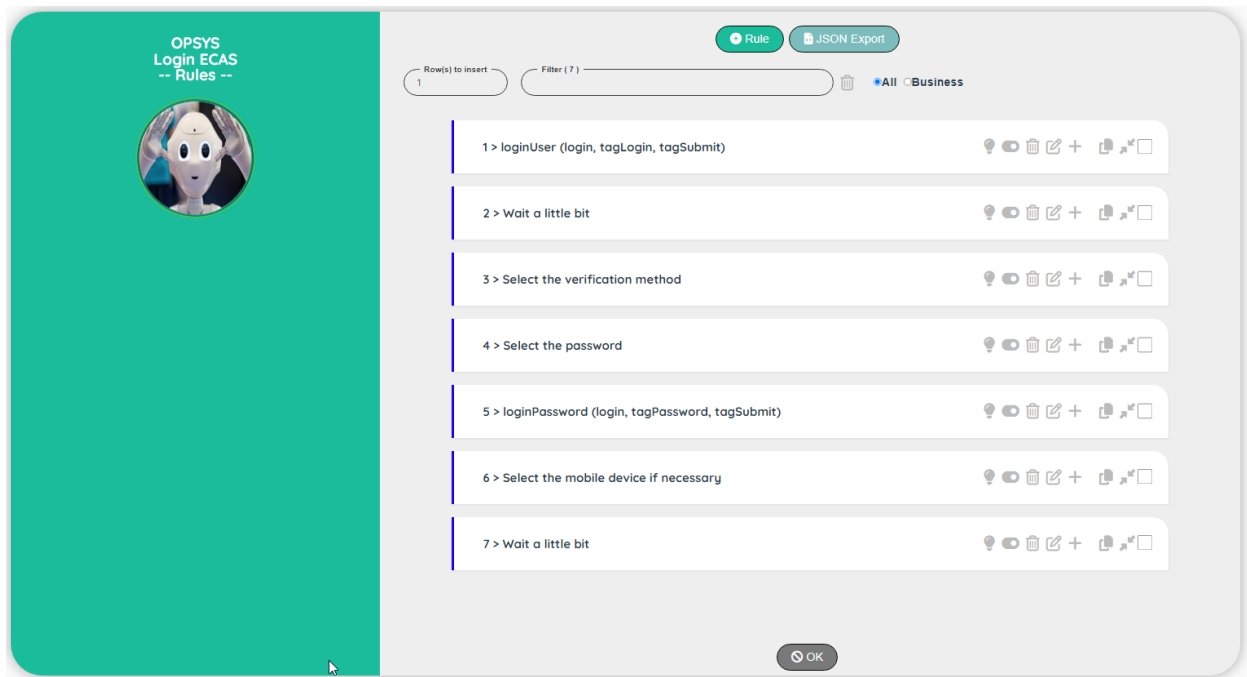
Active

Save

Cancel

Topic	Icon	Comment
Project		For info: Name of the project
Rule set		Code of the rule set
Active		Active, Not active
Save		Save the edit
Cancel		Discard the edit

Rules



Topic	Icon	Comment
Add Rule		Add a rule at the beginning of the list.
JSON Export		Export rules into a .json file
Download		Download the .json file
Active/Inactive		Set the test Inactive/Active
Business/Designer		Set the comment type to Business / Designer (Technical)
Cancel		Back to the rule set.

Note: To view how to upload a '.json' file into an Excel sheet, please refer to the section: [How to process the .json file into Excel](#)

Rules Edit

OPSYS

-- Rule --



Project

OPSYS

Rule

Login ECAS

Continue

Yes

Condition

1 == 1

Result

#loginUser: \$P1, @OPSYS_tagLogin, @OPSYS_tagSubmitLo

Message

Comment

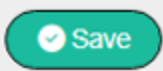
loginUser (login, tagLogin, tagSubmit)

Status

Active

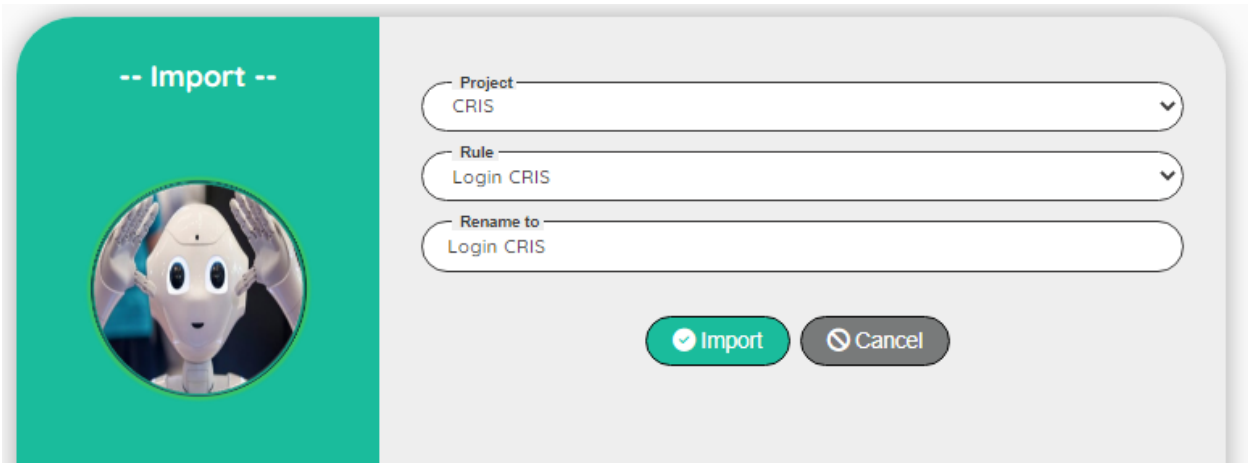
Save

Cancel

Topic	Icon	Comment
Project		For info: Name of the project
Rule set		For info: Name of the rule set
Continue		Select Yes or Skip
Condition		Any valid JavaScript expression
Result		Any valid JavaScript expression or an existing function with its parameters (function starts with #)
Comment	 Comment for the Business  Comment for the Designer	Comment on the test
Active		Active, Not active
Save		Save the edit
Cancel		Discard the edit

Import rule set (including rules)

This screen will allow you to import the rules from another project



Topic	Icon	Comment
Project		Select a project
Rule set		Select a rule
Rename to		If necessary, rename the rule
Import		Import the rules
Cancel		Discard the import

Loop

Principle

The Loops are very useful when you need to repeat a process.
Example: Upload a set of documents, key a set of experts data...

To define a loop, you need to use the instruction Loop



Scenario	Test a simple Loop
Action	Loop
Comment	through a cycle
Max Loop	\$MaxCycle
Status	Active

Loop has a parameter: Max Loop to define the number of loops.

Note: If a 'It block' is defined, the loop is inside the block (and so, depends on the condition of a skipIt or a skipDescribe)

You can control the loop thanks to the variable \$Loop or \$Loop1 (it's equivalent).

For instance, you can enter a value with specific data from the dataset like this:

setValue @Portal_LumpSum, **#\$Dataset_Lump Sum Comment\$Loop** to key data from the dataset:

- **#SEA-2023_Lump Sum Comment1**: This is the first comment
- **#SEA-2023_Lump Sum Comment2**: This is the second comment
- **#SEA-2023_Lump Sum Comment3**: This is the third comment

You can define a nested loop if you define a second Loop inside the first one

In the image above, the line 6 define a nested loop.

If the maximum loop for the first one is 2 and 3 for the second, we will have the following sequence:

(1) → 1.1, 1.2, 1.3 **(2)** → 2.1, 2.2, 2.3

Break a loop

Note: Be careful, the update of the variable used in the parameter of the Loop (Max Loop) during the process has no effect as the number of loops is evaluated during the definition of the loop. If you need to break a rule in progress, you can use a skipIt (but the loop will continue its cycle without any process)

End of a loop

There are three ways to define the end of a loop:

- 1) There is no more step

As in the image above, the loop 1 and the loop 2 will be defined until the line 7

- 2) There is a Describe

When a Describe is detected, the Robot understand that the current loop is ended.

This is the most common way to indicate the end of a loop

- 3) There is an End Loop

When you have a skipIt before the loop, the existence of the loop will depend of the skipIt condition. In the case you have nested loop, you cannot use a Describe to end a loop.

In this case, you need to use End Loop. This instruction will be ignore if a skipIt is triggered.

Examples of loop

A simple case: One loop inside a scenario

Action	Comment	Loop
Describe	Manage the deliveries	
It	Possible to manage deliveries	
Step	Get the number of deliveries from the dataset into the variable \$NbDelivery	
Loop	Through all the deliveries (max loop = \$NbDelivery)	1
Step	Add a new delivery	1
Step	Enter the name from the dataset using #Dataset_Delivery\$Loop	1
It	Possible to continue inside a loop	1
Step	...	1
Describe	Manage the document (End of the loop on the deliveries)	
It		

Two independent loops inside a scenario

Action	Comment	Loop
Describe	Manage the deliveries	
It	Possible to manage deliveries	
Step	Get the number of deliveries from the dataset into the variable \$NbDelivery	
Loop	Through all the deliveries (max loop = \$NbDelivery)	1
Step	Add a new delivery	1
Step	Enter the name from the dataset using #Dataset_Delivery\$Loop	1
It	Possible to continue inside a loop	1
Step	...	1
Describe	Manage the document (End of the loop on the deliveries)	
It	Possible to manage document	
Step	Get the number of documents from the dataset into the variable \$NbDocument	
Loop	Through all the documents (max loop = \$NbDocument)	1
Step	Add a new document	1
Step	Enter the name of the document from the dataset using #Dataset_Document\$Loop	1
It	Possible to continue inside a loop	1
Step	...	1
Describe	Manage the save data (End of the loop on the document)	
It		

Two nested loops inside a scenario

Action	Comment	Loop
Describe	Manage the contractors	
It	Possible to manage contractors	
Step	Get the number of contractors from the dataset into the variable \$NbContractor	
Loop	Through all the contractors (max loop = \$NbContractor)	1
Step	Get the name of the contractor in the variable \$Name	1
It	Possible to continue inside a loop	1
Step	...	1
It	Possible to manage Experts of the contractor	1
Step	Get the number of experts from the dataset into the variable \$NbExpert	1
Loop	Through all the expert (max loop = \$NbExpert)	2
Step	Add a new Expert	2
Step	Enter the name of the expert from the dataset using #Dataset_Expert\$Loop	2
It	Possible to continue inside a loop	2
Step	...	2
Describe	Save the contractor (end of the loop 2, back to loop 1)	1
It	Possible to save the contractor	1
Step	...	1
Describe	Finalise the test (end of the loop 1)	
It	Possible to finalise the test	

Two nested loops inside a scenario with a conditional skipIt (we need to use End Loop)

Action	Comment	Loop
Describe	Manage the contractors	
It	Possible to manage contractors	
Step	Get the number of contractors from the dataset into the variable \$NbContractor	
Loop	Through all the contractors (max loop = \$NbContractor)	1
Step	Get the name of the contractor in the variable \$Name	1
It	Possible to continue inside a loop	1
Step	...	1
It	Possible to manage Experts of the contractor	1
Step	Skip the test if the contractor is 'ArtComputer' (using \$Name)	1
Step	Get the number of experts from the dataset into the variable \$NbExpert	1
Loop	Through all the expert (max loop = \$NbExpert)	2
Step	Add a new Expert	2
Step	Enter the name of the expert from the dataset using #Dataset_Expert\$Loop	2
End Loop	End of the loop 2, back to loop 1	1
It	Possible to save the contractor	1
Step	...	1
Describe	Finalise the test (end of the loop 1)	
It	Possible to finalise the test	

Performance

Principle

The Robot is not designed to measure the performance of an application (like load runner for instance) however, you can measure the elapsed time between two points in the application.

You have also the possibility to compare the last measure with the average of the last 10 tests. It can be useful after a new deployment for instance.

You can set as many timers you want in a scenario by respecting the following rules:

- Each timer as a unique identifier (topic).
- The identifier used during the closure must be equal to the identifier of the start.

In the dashboard, you have the possibility to display the performance of a story or focus on the performance of a specific step in a story.

To measure elapsed time, you can use the following functions:

startTimer	topic		Start a timer to measure the
stopTimer	environment	topic	Store the elapsed time in the database

Define the measure

If your scenario is well structured, it should be very easy to identify the bloc of actions thanks to the “Describe” and the “It”.

If you want to measure the global performance of a test, I suggest to start the measure, just after the login.

Note: If you want to measure the performance of the login, take into account that the measure is done by scenario. You cannot display a global measure of the login of your application!

To indicate the starting point of the measure, use the function `startTimer()` with the name of the topic (a unique identifier). Keep the name short!

Note: The topic is case-insensitive.

To indicate the end of the measure, use the function `stopTimer()` with the parameters: Environment (Example: ACC, TEST or PROD) and the name of the topic used in the `startTimer` function.

Warning: be careful when using the function `pause()` because you will increase artificially the performance.

Also, increasing a pause of an existing scenario will impact the global performance!

Mechanism

The stopTimer will measure the elapsed time and store the value in the database.

The Robot will perform the following tasks:

Shift the 10 existing measures up and store the elapsed time at the measure 11

ID	T1	T2	T3	T4	...	T11	T12	T13	...
1	M1	M1	M1	M1		M1	M2	M3	
2			M2	M2		M2	M3	M4	
3				M3		M3	M4	M5	
4						M4	M5	M6	
5						M5	M6	M7	
6						M6	M7	M8	
7						M7	M8	M9	
8						M8	M9	M10	
9						M9	M10	M11	
10						M10	M11	M12	
11	M1	M2	M3	M4		M11	M12	M13	

With Txx: Timing and Mxx: Measure

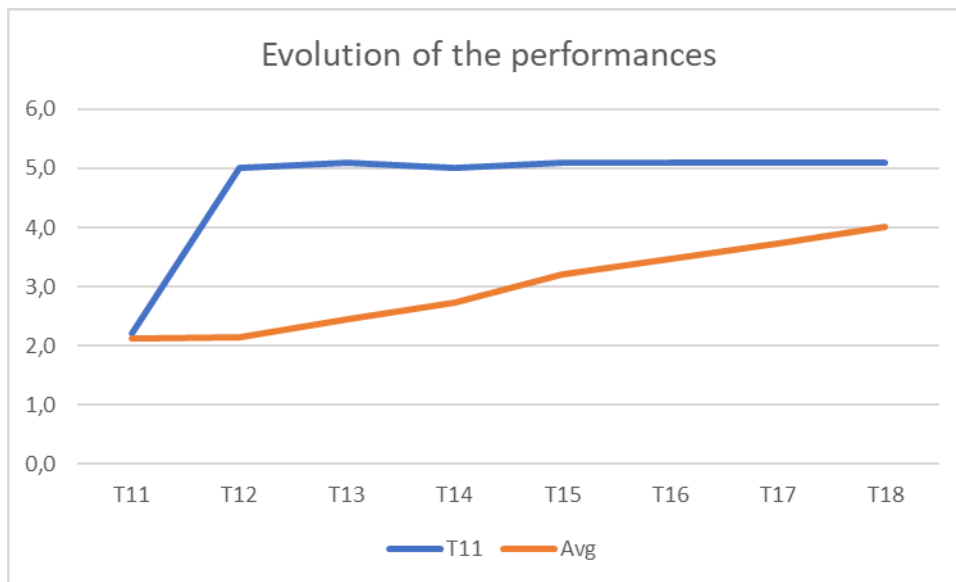
Note: we use a cheat with the first and the second measure, to keep the coherence of the data

Using this mechanism will reduce excessive measurement variations.

However, be careful when you need to perform a measure after a new deployment, if the elapsed time is suddenly high, the impact will not be very visible at first!

Example:

ID	T11	T12	T13	T14	T15	T16	T17	T18
1	2,0	2,1	2,2	2,1	2,2	2,3	2,0	2,1
2	2,1	2,2	2,1	2,2	2,3	2,0	2,1	2,2
3	2,2	2,1	2,2	2,3	2,0	2,1	2,2	2,1
4	2,1	2,2	2,3	2,0	2,1	2,2	2,1	2,2
5	2,2	2,3	2,0	2,1	2,2	2,1	2,2	5,0
6	2,3	2,0	2,1	2,2	2,1	2,2	5,0	5,1
7	2,0	2,1	2,2	2,1	2,2	5,0	5,1	5,0
8	2,1	2,2	2,1	2,2	5,0	5,1	5,0	5,1
9	2,2	2,1	2,2	5,0	5,1	5,0	5,1	5,1
10	2,1	2,2	5,0	5,1	5,0	5,1	5,1	5,1
11	2,2	5,0	5,1	5,0	5,1	5,1	5,1	5,1
Avg	2,1	2,2	2,4	2,7	3,2	3,5	3,7	4,0



To quickly highlight the variation, we need to show on the same graph the average and the last measure.

As you can see on the graph, at T12 we suddenly increase the elapsed time!

Performance with the Robot

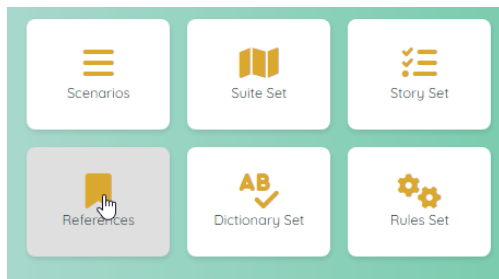
You can visualize the performances of a scenario in the Dashboard.

At the upper left, you can see an icon with a graph, each click will display a different graph.

Dashboard	 Status		Graph of the execution of the story (status)
Dashboard	 Story		Graph of the performance of all the scenarios of the story
Dashboard	 Step		Graph of the performance of all the scenarios of a specific step of a story

Note 1: The step is a kind of zoom on the graph to highlight specific scenarios.

Note 2: In the Dashboard, the Robot can use the value 'Environment' of the reference data. Be sure that you have defined the correct environment before displaying the performance! If no reference 'Environment' can be found, you will not be able to select the performance graph!



PROSPECT
 User: goffipl
 -- Reference --


Project

PROSPECT

Code

Environment

Value

ACC

Comment

Environment for the test

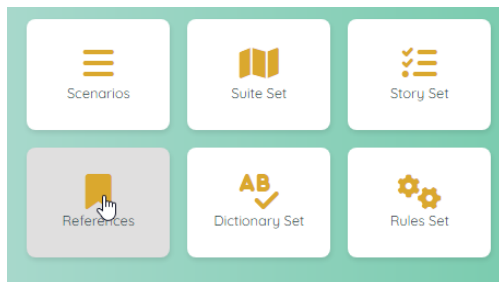
Status

Active

Save

Cancel

Note 3: In the Dashboard, the Robot can use the value 'AllEnvironments' of the reference data. If the reference doesn't exist, there is no error, but the list will be limited to the current environment.



The screenshot shows the 'PROSPECT User: goffipl -- Reference --' form. It includes a user profile icon and a form with the following fields:

- Project: PROSPECT
- Code: AllEnvironments
- Value: ACC,TEST,PROD
- Comment: All the Environments used for the performance
- Status: Active

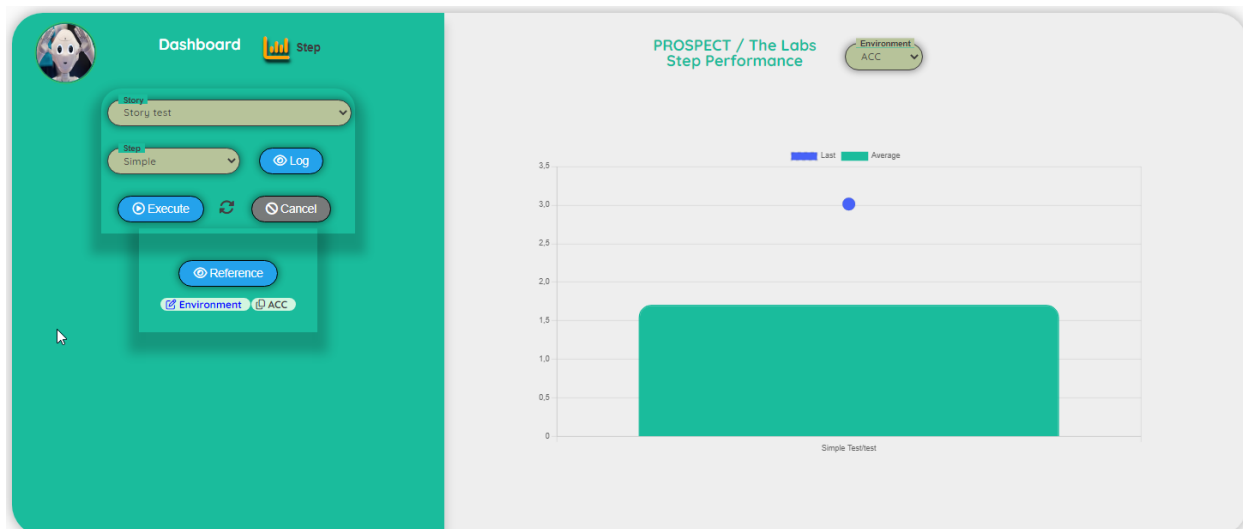
Example: Performance of the story



At this point the selection of a specific step has no impact on the graph.
The first three bars represent the step: 'Test' and the last one, the step: 'Simple'.

Example: Performance of the step.

In this case, we decided to highlight the step: Simple to have a better view of the gap between the average and the last measure.
The step graph will act as a zoom.

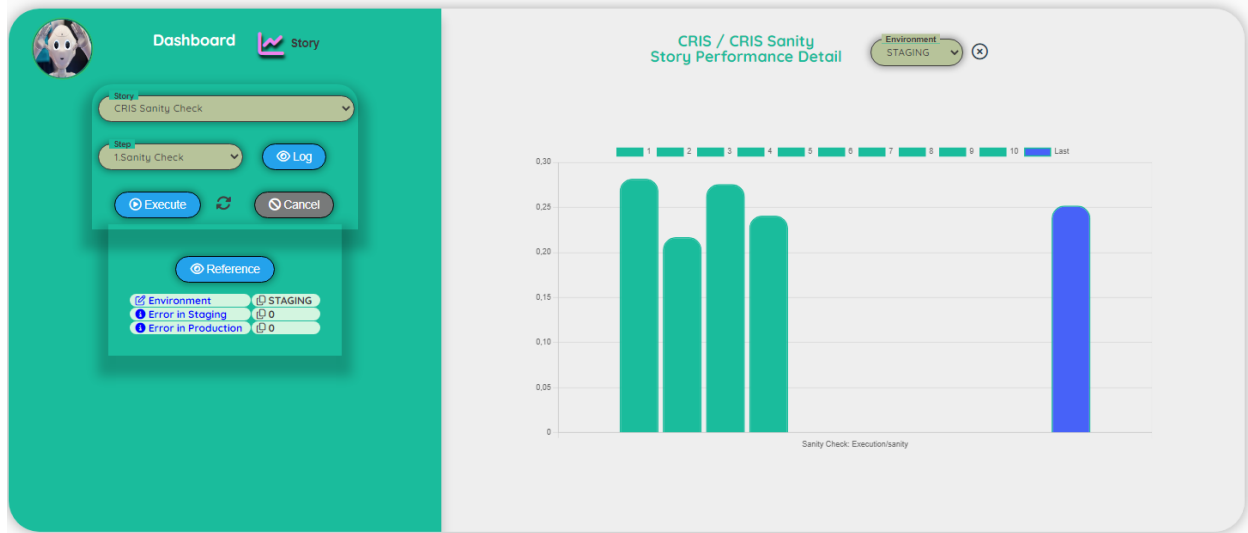


Example: Performance detail (story or step).

This graph is useful to view the variance of the measures.

As we use up to 10 measures to compute the average, it's also important to check the coherence of the measures. Although, the quality of the average will be better if you have already run the test a few time.

From 1 to 10 (in green), the data that will be used for the computation of the average.
In blue the last measure recorded.



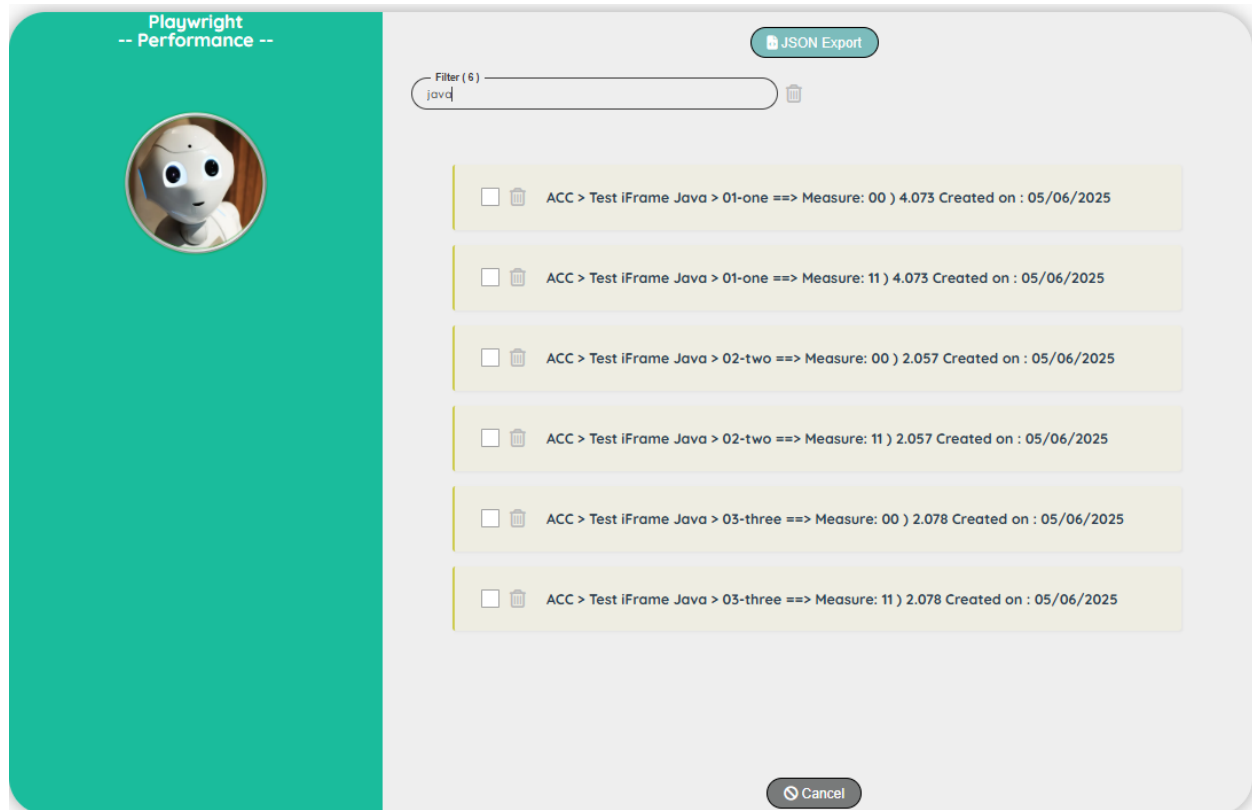
You can change the environment to get a quick comparison of the performances.

In this case the performances in PROD



Performance with Admin right

If you are connected to the application as Admin, you can export the performance data into a Json file.



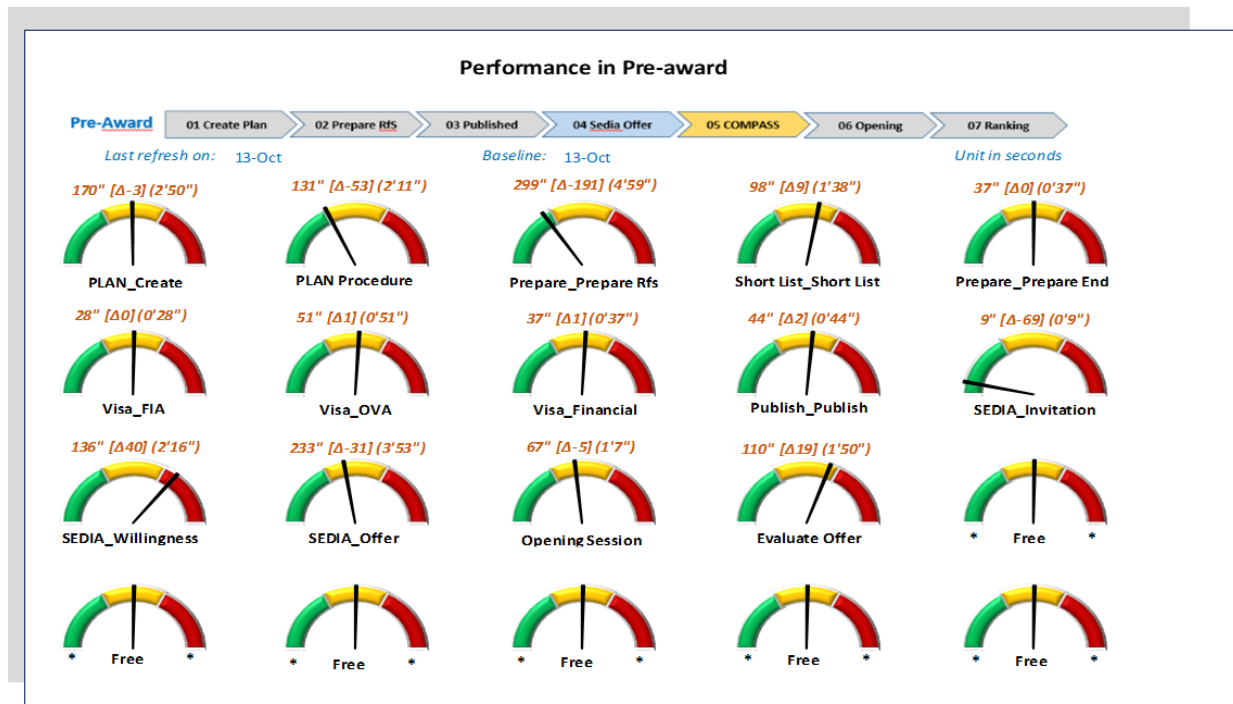
The .Json file has the following structure.

You can see how to import a .Json file into Excel in the Chapter: Test / How to process the .json file into Excel

```
{
  "Performance": [
    {
      "performanceID": 111,
      "projectID": 85,
      "project": "PROSPECT",
      "scenarioID": 105,
      "scenario": "Test",
      "space": "ACC",
      "topic": "fulltest",
      "created": "11/12/2024",
      "sequenceID": 1,
      "sequenceID2": "01",
      "measure": 43.236
    }
  ],
}
```

Example of graphs of performance designed in Excel

This was designed for a previous project to show the evolution of the performances in the different business categories.



Testing mobiles (web browser)

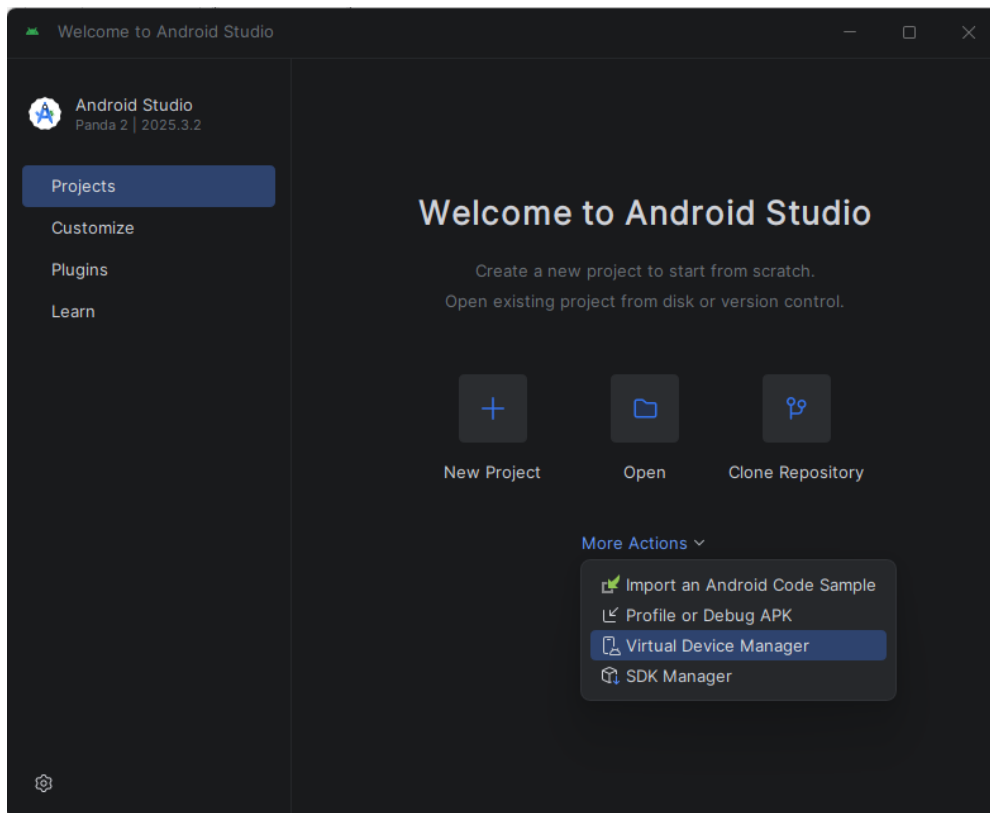
Android Studio - Installation

To test a mobile Android device, you need to carefully setup your environment !

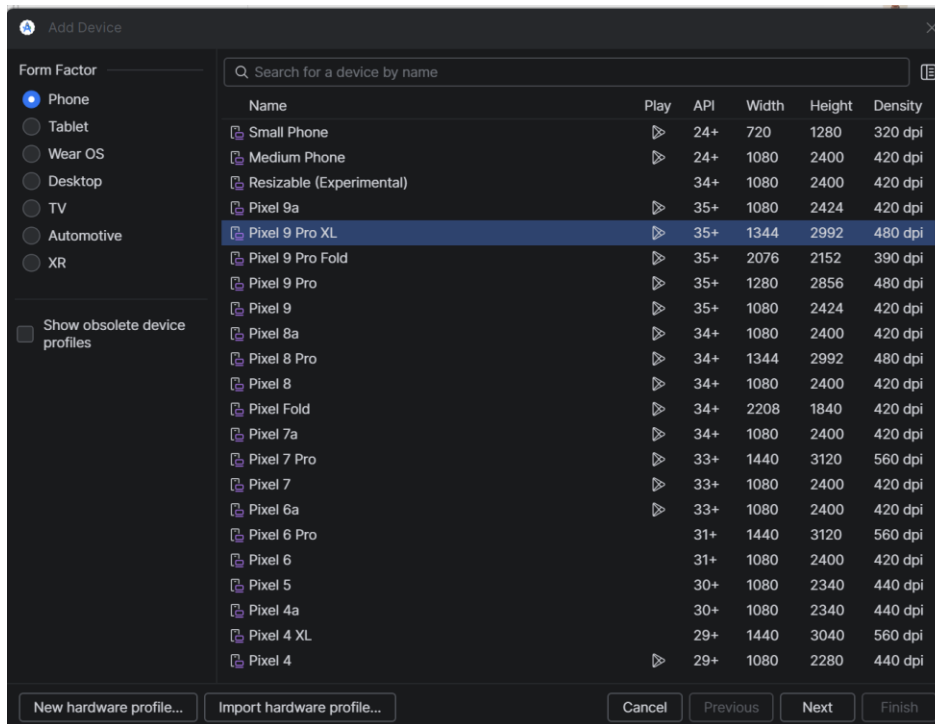
Download: Android Studio: <https://developer.android.com/studio>

Android Studio is a platform for the developers. It offers an emulator for different Android devices like: Phone, Tablet, Wear OS (e.g. watches), Desktop, TV, Automotive, XR.

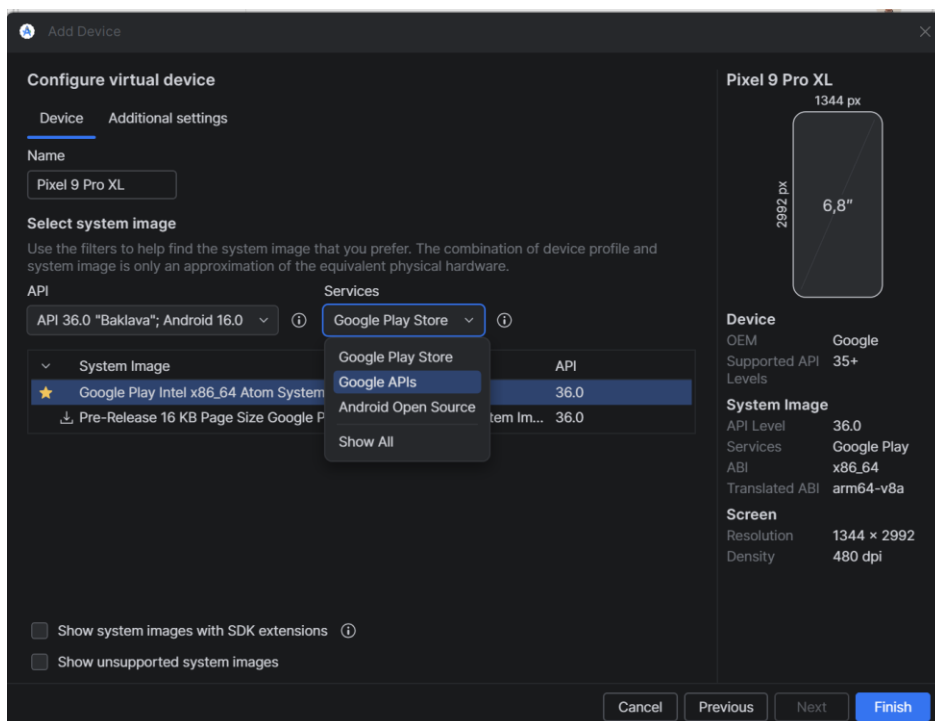
When you have installed Android Studio, click on the menu: More Actions and select “Virtual Device Manager”



Click on the “+” to add a new device and select a phone (e.g. Pixel 9 Pro XL)

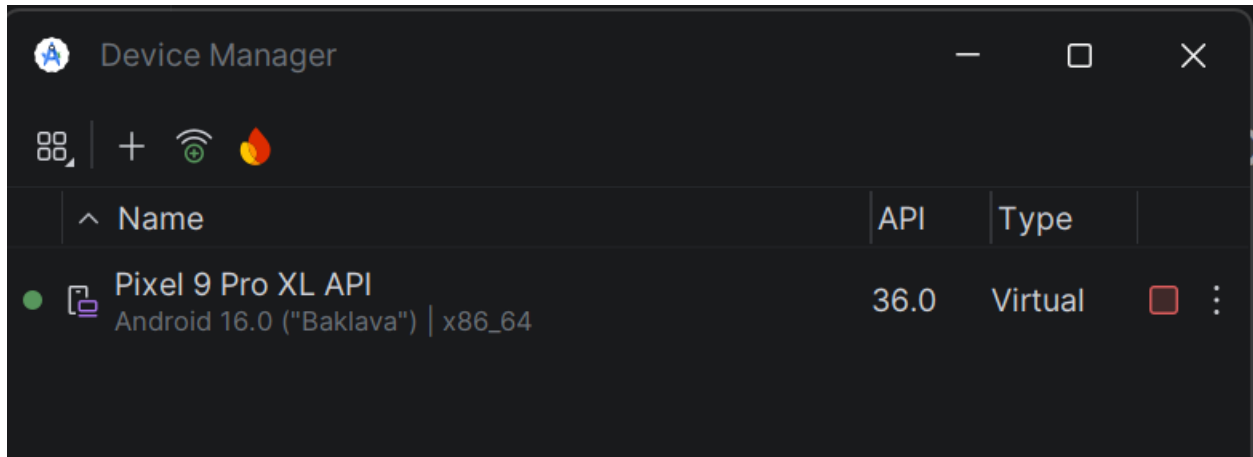


Click on Next, and now a **very important step**: you need to select **Google APIs** rather than Google Play Store

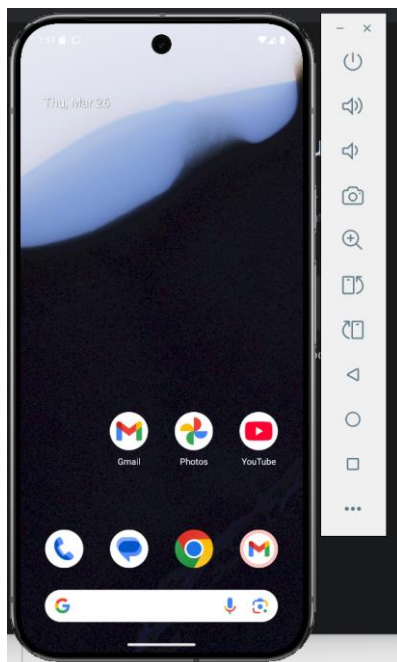


Select the system image: Google APIs.... Atom System Image to enable the button Finish.

This step is very important because the Google Chrome from the Play Store contains a lot of security that will block Playwright from exchanging information (during the handshake protocol).



When you see a green dot next to the selected device, you are ready to go...



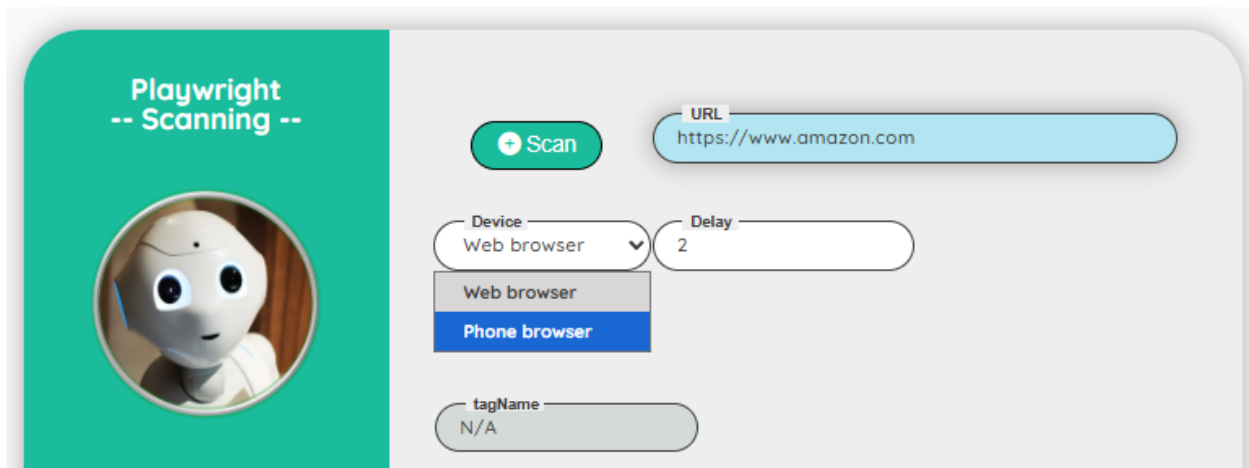
The menu on the right can be used to control the phone:

		Switch On/Off the device
		Increase / decrease the volume of the device
		Take a snapshot
		Zoom on the screen
		Rotate the device portrait to landscape
		Back
		Home
		Overview (?)

The Home button is very important because in a lot of scenarios, you will expect that the device is in the Home position.

Note: this configuration will not work to test Android Apps (in this case, you need to follow instruction on the next section)

In the Robot tool, you can use the **Scan** with the option **Phone browser** to help you to identify an element from your Phone browser.



Testing mobiles (Apps)

If you need to test Android Apps from the Play Store, you need to install a Phone emulator based on Play Store.

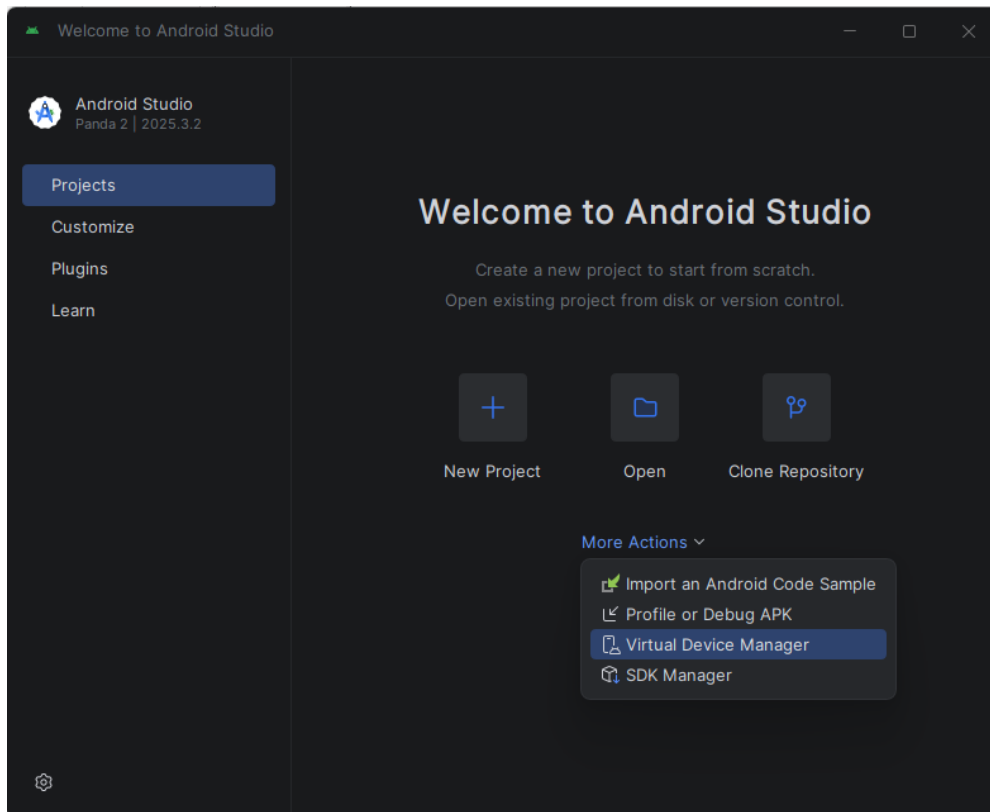
Android Studio - Installation

To test a mobile Android device, you need to carefully setup your environment !

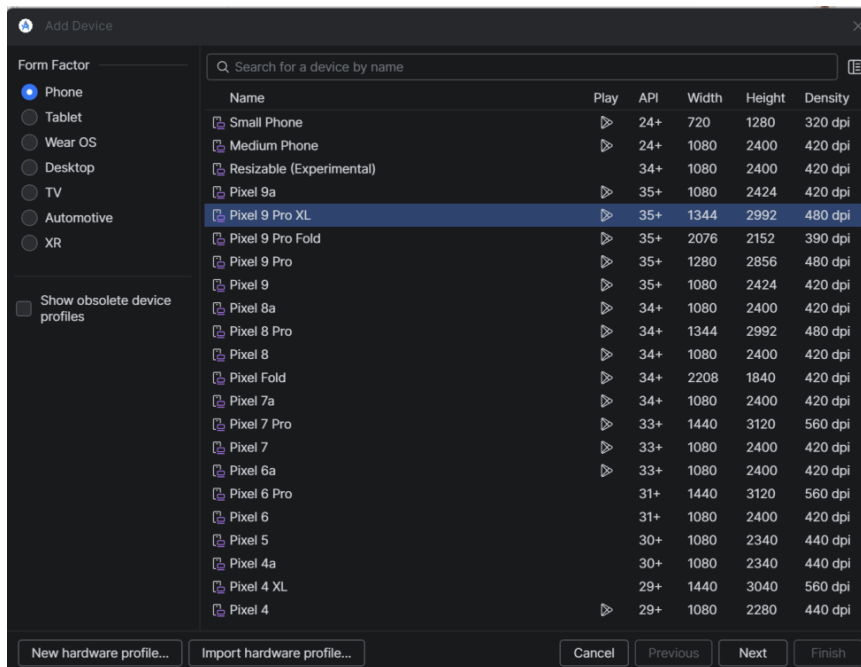
Download: Android Studio: <https://developer.android.com/studio>

Android Studio is a platform for the developers. It offers an emulator for different Android devices like: Phone, Tablet, Wear OS (e.g. watches), Desktop, TV, Automotive, XR.

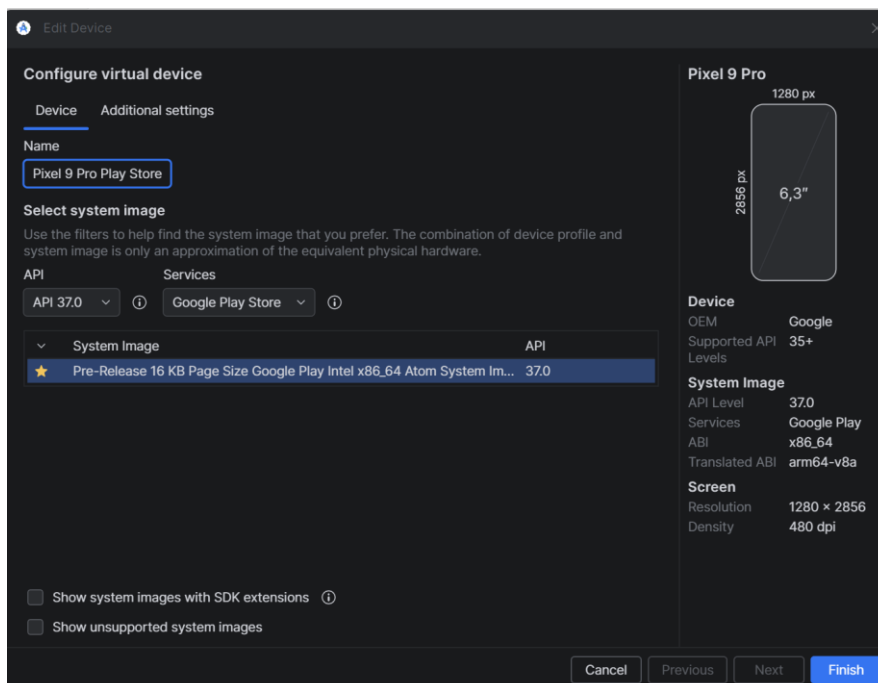
When you have installed Android Studio, click on the menu: More Actions and select “Virtual Device Manager”



Click on the “+” to add a new device and select a phone (e.g. Pixel 9 Pro XL)



Click on Next, and now a **very important step**: you need to select **Google Play Store** rather than Google APIs.



Android Studio - Scanner

Unfortunately, there is not yet an integrated scanner in the Robot tool.

If you want to scan the Apps, you can install node.js on your computer and create the following files:

- Index.html: the user interface where you can select an element with your mouse
- Server.js: REST API to scan the element under the mouse position

In order to play the scanner, you will need to install the following package

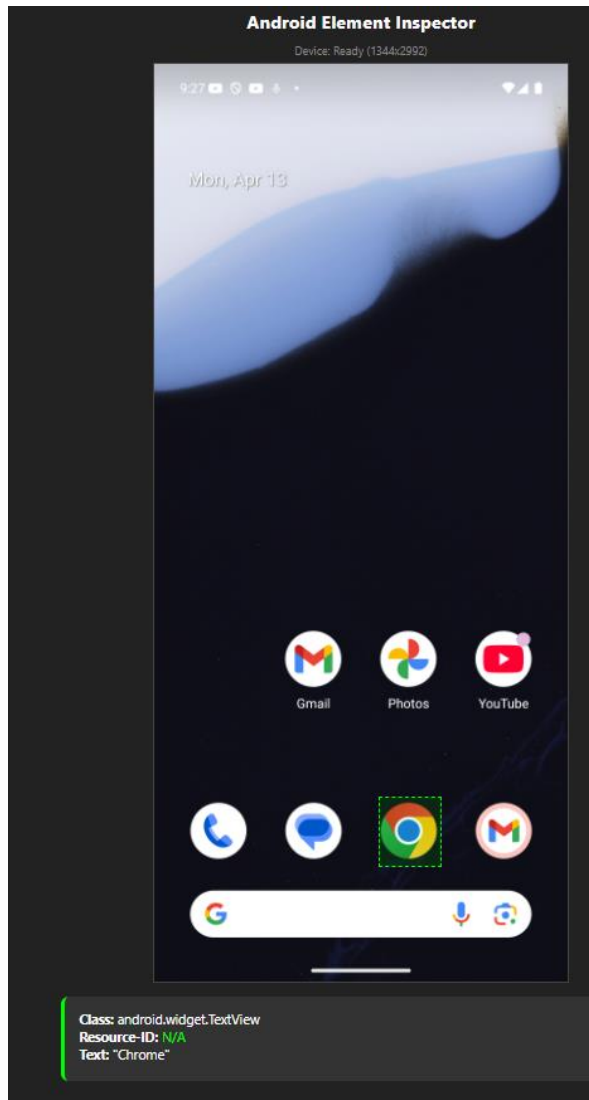
- npm install express
- npm install xml2js

Run the server:

- node server.js

```
PS C:\Apache24\htdocs\AndroidInspector> node server.js
Connected to: sdk_gphone64_x86_64
Scanner running at http://localhost:3000
```

- launch your browser with the address: <http://localhost:3000> and move your mouse over an element (you will see a green border around the selected element).
In the bottom of the screen, you have the properties of the element.
(in the example, we can use the function **phonetap** (Text, Chrome) to simulate the click on the Chrome icon)



Source: server.js

```

const { _android: android } = require('playwright');
const express = require('express');
const xml2js = require('xml2js');
const app = express();
const port = 3000;

let device;
const parser = new xml2js.Parser();

app.use(express.static('public'));

async function initDevice() {
  [device] = await android.devices();
  if (!device) {
    console.error("No device found.");
    process.exit(1);
  }
  console.log(`Connected to: ${device.model()}`);
}

// Helper: Convert Android XML bounds "[x1,y1][x2,y2]" to numeric object
function parseBounds(boundsStr) {
  const m = boundsStr.match(/\[(\d+),(\d+)\]\[(\d+),(\d+)\]/);
  if (!m) return { x: 0, y: 0, width: 0, height: 0 };
  const x1 = parseInt(m[1]);
  const y1 = parseInt(m[2]);
  const x2 = parseInt(m[3]);
  const y2 = parseInt(m[4]);
  return { x: x1, y: y1, width: x2 - x1, height: y2 - y1 };
}

// Helper: Recursively clean the XML tree into a light JSON
function simplifyNode(node) {
  const attr = node.$ || {};
  const bounds = parseBounds(attr.bounds || "");

  //console.log('attr', attr)

  return {
    type: attr.class || "node",
    res: attr['resource-id'] || "",
    text: attr.text || "",
    bounds: bounds,
    children: node.node ? node.node.map(simplifyNode) : []
  };
}

// ... (rest of the code)

async function getScreenSize() {
  try {
    const outputBuffer = await device.shell('wm size');
    const output = outputBuffer.toString(); // <--- Convert Buffer to String

    console.log("Device Output:", output); // Helpful for debugging

    const match = output.match(/(\d+)x(\d+)/);
    if (match) {
      return {

```

```

        width: parseInt(match[1]),
        height: parseInt(match[2])
    };
    }
} catch (e) {
    console.error("Failed to get screen size, using fallback", e);
}
return { width: 1080, height: 1920 }; // Fallback
}

app.get('/snapshot', async (req, res) => {
    try {
        // 1. Get screenshot first
        const screenshot = await device.screenshot();

        // 2. Get screen resolution
        const size = await getScreenSize();

        // 3. Dump UI.
        // Note: Sometimes uiautomator dump fails if the screen is transitioning.
        // We wrap this in a try/catch so we at least get the image if the XML fails.
        let tree = null;
        try {
            await device.shell('uiautomator dump /data/local/tmp/view.xml');
            const xmlBuffer = await device.shell('cat /data/local/tmp/view.xml');
            const xmlData = xmlBuffer.toString(); // <--- Convert Buffer to String

            if (xmlData.includes('<?xml')) {
                const result = await parser.parseStringPromise(xmlData);
                const rootNode = result.hierarchy.node ? result.hierarchy.node[0] : null;
                if (rootNode) {
                    tree = simplifyNode(rootNode);
                }
            }
        } catch (xmlError) {
            console.warn("XML Dump failed, continuing with image only");
        }
    }

    res.json({
        image: screenshot.toString('base64'),
        tree: tree,
        screenSize: size
    });
} catch (e) {
    console.error("Snapshot Error:", e);
    res.status(500).send(e.message);
}
});

app.get('/click', async (req, res) => {
    try {
        const x = Math.round(req.query.x);
        const y = Math.round(req.query.y);

        if (isNaN(x) || isNaN(y)) {
            return res.status(400).send("Invalid coordinates");
        }

        console.log(`Tapping at: ${x}, ${y}`);

        // Use shell instead of device.tap to avoid the BySelector NullPointerException
        await device.shell(`input tap ${x} ${y}`);

        res.send("Clicked");
    }

```

```
} catch (e) {  
  console.error("Click error:", e);  
  res.status(500).send(e.message);  
}  
});  
  
initDevice().then(() => {  
  app.listen(port, () => console.log(`Scanner running at http://localhost:${port}`));  
});
```

Source: index.html

```
<!DOCTYPE html>  
<html>  
  
<head>  
  <title>Android Inspector</title>  
  <style>  
    body {  
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;  
      background: #222;  
      color: white;  
      display: flex;  
      flex-direction: column;  
      align-items: center;  
      margin: 0;  
    }  
  
    h1 {  
      margin: 10px 0;  
      font-size: 1.2rem;  
    }  
  
    #container {  
      position: relative;  
      line-height: 0;  
      border: 2px solid #444;  
      background: #000;  
    }  
  
    #screen {  
      height: 80vh;  
      cursor: crosshair;  
    }  
  
    #highlighter {  
      position: absolute;  
      border: 2px dashed #00ff00;  
      background: rgba(0, 255, 0, 0.1);  
      pointer-events: none;  
      display: none;  
      z-index: 10;  
    }  
  
    #info-box {  
      width: 90%;  
      max-width: 600px;
```

```

    margin-top: 15px;
    padding: 15px;
    background: #333;
    border-radius: 8px;
    min-height: 60px;
    font-size: 14px;
    border-left: 5px solid #00ff00;
    word-wrap: break-word;
  }

  .status {
    color: #888;
    font-size: 12px;
    margin-bottom: 5px;
  }
</style>
</head>

<body>
  <h1>Android Element Inspector</h1>
  <div id="status" class="status">Connecting to device...</div>

  <div id="container">
    <img id="screen" src="" />
    <div id="highlighter"></div>
  </div>

  <div id="info-box">
    Move your mouse over the screen to inspect elements.
  </div>

  <script>
    // 1. DECLARE GLOBAL VARIABLES (Required to fix ReferenceError)
    let currentTree = null;
    let screenSize = null;
    let isFetching = false;

    const img = document.getElementById('screen');
    const highlighter = document.getElementById('highlighter');
    const infoBox = document.getElementById('info-box');
    const statusDiv = document.getElementById('status');

    // 2. FETCH DATA FROM SERVER
    async function refresh() {
      if (isFetching) return;
      isFetching = true;

      try {
        const res = await fetch('/snapshot');
        if (!res.ok) throw new Error("Server error");

        const data = await res.json();

        // Update Globals
        img.src = 'data:image/png;base64,' + data.image;
        currentTree = data.tree;
        screenSize = data.screenSize;

        statusDiv.innerText = `Device: Ready (${screenSize.width}x${screenSize.height})`;
      } catch (e) {
        console.error("Refresh failed", e);
        statusDiv.innerText = "Error: Cannot reach server. Is server.js running?";
      } finally {
        isFetching = false;
      }
    }
  </script>

```

```

}

// 3.A MOUSE MOVE HANDLER
img.addEventListener('mousemove', (e) => {
  // Check if we have data yet
  if (!currentTree || !screenSize) return;

  const rect = img.getBoundingClientRect();

  // Scaling factors
  const scaleX = screenSize.width / rect.width;
  const scaleY = screenSize.height / rect.height;

  // Mouse coords to phone coords
  const x = (e.clientX - rect.left) * scaleX;
  const y = (e.clientY - rect.top) * scaleY;

  // Recursive search for the element
  const element = findElementAt(currentTree, x, y);

  if (element && element.bounds) {
    const b = element.bounds;

    // Show highlighter
    highlighter.style.display = 'block';
    highlighter.style.left = (b.x / scaleX) + 'px';
    highlighter.style.top = (b.y / scaleY) + 'px';
    highlighter.style.width = (b.width / scaleX) + 'px';
    highlighter.style.height = (b.height / scaleY) + 'px';

    console.log('element', element)
    // Show info
    infoBox.innerHTML = `
      <strong>Class:</strong> ${element.type}<br>
      <strong>Resource-ID:</strong> <span style="color:#00ff00">${element.res || 'N/A'}</span><br>
      <strong>Text:</strong> "${element.text || ''}"<br>
    `;
  } else {
    highlighter.style.display = 'none';
  }
});

// 3.B. MOUSE CLICK
img.addEventListener('click', async (e) => {
  if (!screenSize) return;
  const rect = img.getBoundingClientRect();
  const x = (e.clientX - rect.left) * (screenSize.width / rect.width);
  const y = (e.clientY - rect.top) * (screenSize.height / rect.height);

  // You'll need a /click endpoint in server.js
  await fetch(`/click?x=${Math.round(x)}&y=${Math.round(y)}`);
  //await fetch('/click');
  refresh(); // Refresh screen after click
});

// 4. FIND ELEMENT LOGIC
function findElementAt(node, x, y) {
  let found = null;
  if (!node || !node.bounds) return null;

  const b = node.bounds;
  // Check if coordinates are inside this node
  if (x >= b.x && x <= (b.x + b.width) && y >= b.y && y <= (b.y + b.height)) {
    found = node;
    // Check children for a more specific match
  }
}

```

```
    if (node.children && node.children.length > 0) {  
      for (const child of node.children) {  
        const deeper = findElementAt(child, x, y);  
        if (deeper) {  
          found = deeper;  
        }  
      }  
    }  
  }  
  return found;  
}  
  
// Initial load and loop  
refresh();  
setInterval(refresh, 3000);  
</script>  
</body>  
</html>
```